

Multi-node: the lease, the clock, and the fence

The win: you can explain, to an interviewer or to yourself, why a lease is a lock with a time on it and why that time may only be read from your own monotonic clock counting from the moment you sent; why a Lua script is the unit of atomicity in Redis and what breaks when the token is issued outside it; why heartbeats may route a peer but never decide who owns a room, and why a dead owner's lease is waited out rather than stolen; why a process paused past its lease cannot be stopped by any check it performs itself, and what Kleppmann's fencing token is; why the fence must sit in the same SQL statement as the write, and what `FOR UPDATE` does there; where this design sits against ZooKeeper, etcd and Redisson, and what it gives up under CAP; and you can point at the line of code for every claim.

Verification status, honestly. Every commit in this chapter passed lint, the type checker and the unit tests (holder parsing, keepLease, the routing table and router, takeLease, watchFence, peer-client's reroute on 4409, the config fields), and each guard was mutation-checked. The integration suites (packages/lease index.int, api session.int, sync one-room-globally.int, one-room-join-race.int, failover.int, packages/db fence.int including the 20-round real race and the naive control) and the three chaos runs (kill-owner-failover, fenced zombie, fenced partition) are written and committed and *have not been run*: the host was too loaded to run Docker suites beside other work. They run on build/epics before anything merges to main. Where this page says "measured", it was measured in learning-tests/redis or an earlier epic; where it says "the test asserts", the test exists and is unrun. Section 8 is the by-hand demo to run first.

How to use this page. Every code block is cut from the repo at commit f21d6d2, so what you read is what is committed. Sections 2, 4 and 5 are the ones a distributed-systems interview will dig into; section 6 is the comparison you will be asked to make.

```
./init.sh # now two sync nodes
(3001, 3003) behind one Redis, through toxiproxy
pnpm exec vitest run packages/lease/src/lease.test.ts apps/sync/src/fence.test.ts packages/peer-client/src/peer.test.ts # the pure halves
docker compose exec -T redis redis-cli --no-auth-warning GET lease:<doc> #
"1:sync": token 1, owned by node sync
```

Try it, once the stack is up: section 8 walks the kill, the pause and the partition by hand, then the suites and the chaos runs, with the exact commands and what each should show.

The map: what multi-node means here

- 1 · The lease: one script, one token, three compare-and-sets
- 2 · Clocks against tokens: when a holder must stop believing itself
- 3 · One room globally: the loser says 4409, and /session knows the owner
- 4 · Failover: heartbeats as a hint, and a dead lease waited out
- 5 · The fence: the check and the write are one statement

- 6 · Trade-offs and comparisons: where this design sits
 - 7 · Testing two nodes without Docker, and then with it
 - 8 · Two nodes by hand: what to run once the stack is up
 - 9 · What the type system could not see, epic 7 edition
- Your turn: three drills
- Recall, then test yourself

The map: what multi-node means here

Epic 6 made one sync process trustworthy: an acknowledged edit survives a crash. Epic 7 asks what happens when there are *two* of them. The whole design rests on lesson 2's first rule: one room per document orders the ops, and there can never be two rooms for one document, because two rooms would mean two orderings and two documents. So with two processes the question is not "how do they share a room" but "how do they agree, at every moment, on *which one* has it", including the moments when one of them is dead, frozen, or cut off, and cannot be asked. Three features answer it. **F20, one room globally:** every peer of a document lands in the same room, whichever node it asked first. **F21, failover:** kill the node that owns a room and the room moves to the other within one lease, with F18's guarantees intact. **F22, fencing:** a node paused past its lease and then resumed cannot append to the journal; it closes its sockets; the journal has no gap and no duplicate.

STEP	WHERE	AND	WHAT HAPPENS
1	packages/lease/src/index.ts	packages/lease/src/lease.ts	a lease per document in Redis: acquire, renew and release as three Lua scripts; a fencing token from <code>INCR</code> issued with the <code>SET</code> ; a <code>Holder</code> parsed back from <code>"<token>:<node>"</code>
2	packages/lease/src/lease.ts		<code>keepLease</code> : when a holder must stop believing it holds the lease, on its own monotonic clock, counting from the moment a renewal was <i>sent</i>
3	apps/sync/src/server.ts	apps/api/src/app.ts	a room opens only under the lease; the loser closes its peers with 4409; <code>/session</code> answers the owner's address from a routing table <code>id=url,id=url</code> ; the worker routes the same way
4	packages/lease/src/lease.ts	scripts/chaos/kill-owner-failover.ts	heartbeats as a routing hint; <code>takeLease</code> waits a dead owner's lease out and never steals it; the chaos run kills the owner and does not restart it
5	packages/db/migrations/0014_fence.sql	packages/db/src/index.ts	the fence: <code>claim</code> the document under the lease token before reading the journal; every append is one statement that checks the claim and locks the row; <code>Fenced</code> drops the room

6	apps/sync/src/fence.ts	scripts/chaos/fenced.ts	a fenced append ends the room, not read-only; two chaos runs: docker pause past the lease, and a toxiproxy cut between one node and Redis
---	------------------------	-------------------------	---

Four ideas carry the epic, and they are the four an interviewer will ask about. **A lease is a lock with an expiry** (section 1): a holder that dies stops renewing, and the lock frees itself; the price is that "I hold it" is now a statement about time. **Only your own clock, and only the moment you sent** (section 2): a holder trusts its lease until $sent + 0.9\ ttl$ on a monotonic clock, never a wall clock, never Redis's clock, and gives up its peers when that passes. **Liveness is a hint, never a truth** (sections 3 and 4): heartbeats decide where a peer is *sent*; the lease alone decides who *owns*; a dead owner's lease is waited out, never stolen. **The lock is not the safety; the storage is** (section 5): a process frozen past its lease cannot know it, so the journal append itself refuses a writer whose claim is no longer the document's, in the same statement that writes. Kleppmann's fencing-token argument is the spine of the chapter, and the repo's own `learning-tests/redis` reproduced the zombie before a line of the epic was written.

THIS REPO	JAVA WORLD	PYTHON WORLD
a Redis lease per document, three Lua scripts, an <code>INCR</code> token; renew every <code>ttl/3</code>	Redisson <code>RLock</code> (Lua too, a watchdog renews every <code>ttl/3</code>); Curator's <code>InterProcessMutex</code> over ZooKeeper ephemeral nodes	<code>redis.lock.Lock</code> with a token and a Lua release; <code>kazoo.Lock</code> ; <code>python-etcd3</code> leases
the token checked by the storage in the write statement	Kafka's producer epoch in the broker; ZooKeeper's <code>zxid</code> ; Chubby's sequencer handed to the file server	the same idea; nothing in the standard library does it for you
routing through <code>/session</code> and a table in one env variable	a service registry (Eureka, Consul) plus sticky routing; or ZooKeeper's ephemeral nodes as the registry	Consul, or a Redis hash of owners
liveness as a heartbeat key with a short <code>PX</code>	ZooKeeper sessions and ephemeral nodes; etcd leases with keep-alive	etcd3 lease keep-alive; Consul checks

1 • The lease: one script, one token, three compare-and-sets

Redis was already in the stack for queues (lesson 3). SPEC §2a fixed the lease's shape from the learning test: *SET NX PX plus a fencing token from INCR in the same Lua script; renew and release are compare-and-set in Lua*. The package that implements it is small on purpose: `index.ts` talks to Redis, `lease.ts` is pure and holds every decision (parsing, deadlines, routing, waiting), so the decisions have unit tests and the I/O has an integration test against the real server.

FROM `packages/lease/src/index.ts`

```
/**
 * Room leases in Redis (SPEC §2a): who owns each document's room, for how long, under which fencing token.
 * Keys: '<prefix>lease:<documentId>' holds "<token>:<nodeId>" with a PX expiry; '<prefix>lease-token:<documentId>'
 * is the document's token counter, never expiring, so a later owner always gets a larger token.
 * '<prefix>node:<nodeId>' is a sync node's heartbeat (E7.2): it expires a third of a ttl after the node's
 * last
 * beat, so a killed owner is known dead long before its leases expire. Routing reads it; nothing safe does.
 * The counter lives in Redis, so a flushed Redis would start the tokens again at 1: an acquire therefore
```

```

takes a
* `floor` (the largest token the journal's fence has seen, E7.3) and never issues a token at or below it.
*/
export type Leases = {
  /** How long a lease lives without renewal. The holder renews every third of it. */
  readonly ttlMs: number;
  /**
   * Take the room if nobody holds it, under a token above `floor`. Answers the holder either way: compare its
   * nodeId (and token) with yours.
   */
  acquire(documentId: string, nodeId: string, floor?: number): Promise<{ acquired: boolean; holder: Holder }>;
  /** Extend, only if `holder` is still exactly the lease's value. False: it expired or is someone else's. */
  renew(documentId: string, holder: Holder): Promise<boolean>;
  /** Let go, only if still ours: a late release must never delete the NEXT owner's lease. */
  release(documentId: string, holder: Holder): Promise<void>;
  owner(documentId: string): Promise<Holder | undefined>;
  /** "This node is alive" for a third of a ttl. A sync node calls it every tenth of one. */
  beat(nodeId: string): Promise<void>;
  /** Which of these nodes beat recently. */
  alive(nodeIds: readonly string[]): Promise<ReadonlySet<string>>;
  /** Resolves once Redis answered a PING; rejects after `timeoutMs`. */
  ready(): Promise<void>;
  close(): Promise<void>;
};

```

FROM `packages/lease/src/index.ts`

```

// One script, so nothing can happen between "nobody holds it", the token and the SET. The token is issued
// only when the SET happens: N racing acquirers of a free room move the counter by exactly one.
const ACQUIRE = `
local current = redis.call('GET', KEYS[1])
if current then return {0, current} end
if tonumber(redis.call('GET', KEYS[2]) or '0') < tonumber(ARGV[3]) then redis.call('SET', KEYS[2], ARGV[3])
end
local holder = redis.call('INCR', KEYS[2]) .. ':' .. ARGV[1]
redis.call('SET', KEYS[1], holder, 'PX', ARGV[2])
return {1, holder}`;
// Compare-and-extend and compare-and-delete: a plain PEXPIRE or DEL would touch a lease that expired and was
// taken by another node in the meantime (reproduced in learning-tests/redis, finding 2).
const RENEW = `if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('PEXPIRE', KEYS[1], ARGV[2])
else return 0 end`;
const RELEASE = `if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else return 0
end`;

```

Read `ACQUIRE` as one atomic step, because that is what Redis makes it. Redis runs commands on one thread, and a Lua script is one command: nothing from any other client runs between its first line and its last. So `GET` (is it held?), the counter `floor`, `INCR` (issue a token) and `SET ... PX` (take it, with an expiry) happen with no gap, and the two properties the tests demand follow. **Exactly one of N racing acquirers wins**, because only the first to run sees no value. **The counter moves by exactly one per acquisition**, because the token is issued only on the branch that also does the `SET`; a loser returns the current holder and touches nothing. That second property is why the join-race test can assert `token: 1` after ten simultaneous joins and `2` after the room closes and is taken again. The naive version, `INCR` then `SET NX` as two round trips, would burn a token per loser and could hand a *smaller* token to a *later* owner if two acquirers interleaved, which section 5's fence could not survive.

`RENEW` and `RELEASE` compare the exact value "`<token>:<node>`" before touching the key. The learning test reproduced why: a holder whose lease had expired issued a plain `DEL` and deleted *another* client's freshly acquired lease. A bare `PEXPIRE` has the same flaw in the other direction: it would extend a lease that is now someone else's. Comparing the whole holder, not just the node id, also stops a node that took the same room twice from renewing its own earlier, dead acquisition (the integration test's "same node, another acquisition" line).

FROM learning-tests/redis/test.ts

```
// 2. CAS renew/release via Lua vs. naive DEL:
//   CONFIRMED. A Lua script that does `if GET(key) == myValue then
//   PEXPIRE/DEL else return 0` correctly refuses to touch a lease it does
//   not own (imposter extend/release both returned 0, real owner's lease
//   was untouched). Demonstrated the actual bug too: a lease holder (C)
//   that lets its lease expire and then issues a bare `DEL key` (with no
//   ownership check) deleted a *different* client's (D's) freshly acquired
//   lease on the same key. assumed "plain DEL is dangerous", actual:
//   reproduced exactly -- D's live lease was wiped by C's stale release.
//
```

FROM packages/lease/src/index.ts

```
export function createLeases({ redisUrl, ttlMs = 10_000, prefix = "", timeoutMs = 1000 }: { redisUrl: string;
ttlMs?: number; prefix?: string; timeoutMs?: number }): Leases {
  // Every call must FAIL within a deadline when Redis is away (E3.1 lesson 1): no offline queue, a command
  // timeout, one retry. A caller that cannot ask refuses to open the room; it never guesses.
  const redis = new Redis(redisUrl, { enableOfflineQueue: false, commandTimeout: timeoutMs,
maxRetriesPerRequest: 1, connectTimeout: timeoutMs });
  // Never log the error object: a failed AUTH carries the password in err.command.args.
  redis.on("error", (err: unknown) => process.stderr.write(`${JSON.stringify({ level: "warn", source: "lease",
message: err instanceof Error ? err.message || err.name : "unknown" })}\n`));
  const leaseKey = (documentId: string): string => `${prefix}lease:${documentId}`;
  const ready = (): Promise<void> => new Promise((resolve, reject) => {
    if (redis.status === "ready") { resolve(); return; }
    const timer = setTimeout(() => { reject(new Error(`redis did not answer within ${String(timeoutMs)} ms`));
}, timeoutMs);
    redis.once("ready", () => { clearTimeout(timer); resolve(); });
  });
  return {
    ttlMs,
    async acquire(documentId, nodeId, floor = 0) {
      const [taken, value] = (await redis.eval(Acquire, 2, leaseKey(documentId), `${prefix}lease-
token:${documentId}`, nodeId, String(ttlMs), String(floor))) as [number, string];
      const holder = parseHolder(value);
      if (!holder) throw new Error(`lease of ${documentId} holds a value that is not a holder`);
      return { acquired: taken === 1, holder };
    },
    renew: async (documentId, holder) => (await redis.eval(Renew, 1, leaseKey(documentId),
formatHolder(holder), String(ttlMs))) === 1,
    async release(documentId, holder) {
      await redis.eval(Release, 1, leaseKey(documentId), formatHolder(holder));
    },
  },
}
```

Three TypeScript things in `createLeases`. **The client options are the E3.1 lesson again:** `enableOfflineQueue: false`, a `commandTimeout` and one retry, so every call *fails within a deadline* when Redis is away; a lease library that hangs is a room that never opens and never says why (the integration test times every failure). **`redis.eval(...)` as `[number, string]`** is a type assertion, not a conversion: `ioredis` types `eval`'s result as `unknown` because a script can return anything, and the cast records what *this* script returns. `parseHolder` then checks the string at runtime, so a key that holds something that is not ours reads as "nobody" rather than as a crash. **The error handler never logs the error object:** a failed `AUTH` carries the password in `err.command.args`, so only `err.message` goes to `stderr`.

FROM packages/lease/src/lease.ts

```
/**
 * Who owns a document's room: a sync node, and the fencing token its acquisition was issued. The token
 * rises with every acquisition of that document (INCR), so two holders are never confused, even when one
 * node takes the same room twice, and a later owner always carries the larger number (the fence of E7.3).
 */
export type Holder = { token: number; nodeId: string };
```

```

/** A node id is a URL path segment and a compose service name: lowercase, digits, dashes. */
export const NodeId = z.string().regex(/^([a-z0-9][a-z0-9-]{0,31})$/, "a sync node id is 1-32 of a-z, 0-9 and -, starting with a letter or digit");

export const formatHolder = ({ token, nodeId }: Holder): string => `${String(token)}:${String(nodeId)}`;

/** A value read back from Redis. Anything else there is not ours to trust: undefined. */
export function parseHolder(value: string): Holder | undefined {
  const match = /^([1-9]\d{0,15}):(.+)/.exec(value);
  if (!match?.[1] || !match[2] || !NodeId.safeParse(match[2]).success) return undefined;
  return { token: Number(match[1]), nodeId: match[2] };
}

```

`Holder` is two numbers of information: *who*, and *which acquisition*. The token is the second one, and the whole of section 5 is built on it rising. `parseHolder` is strict by design: the regex refuses a leading zero, a token of more than sixteen digits (past `Number`'s safe range) and any node id that `NodeId` would refuse. A node id is a URL path segment and a compose service name, so it is lowercase letters, digits and dashes, and Zod's `.regex` carries that rule into every config file that names one.

FROM `packages/lease/src/index.int.test.ts`

```

test("N simultaneous acquires of a free room: exactly one wins, and the token counter moves by exactly one",
  async () => {
    await Promise.all(nodes.map((each) => each.ready()));
    const documentId = randomUUID();
    const results = await Promise.all(nodes.map((each, i) => each.acquire(documentId, `node-${String(i)}`)));
    const winners = results.filter((result) => result.acquired);
    expect(winners).toHaveLength(1);
    // Every loser was told who won, with the winner's token.
    for (const result of results) expect(result.holder).toEqual(winners[0]?.holder);
    expect(winners[0]?.holder.token).toBe(1);
    expect(await raw.get(`${prefix}lease-token:${documentId}`)).toBe("1");
    expect(await a.owner(documentId)).toEqual(winners[0]?.holder);
  });

```

FROM `packages/lease/src/index.int.test.ts`

```

test("a late release by an old holder never deletes the next owner's lease", async () => {
  const documentId = randomUUID();
  const old = await a.acquire(documentId, "sync");
  await new Promise((resolve) => setTimeout(resolve, TTL + 100));
  const next = await b.acquire(documentId, "sync-2");
  await a.release(documentId, old.holder);
  expect(await a.owner(documentId)).toEqual(next.holder);
  await b.release(documentId, next.holder);
  expect(await a.owner(documentId)).toBeUndefined();
});

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
a Lua script as a template literal, sent with <code>redis.eval(script, numKeys, ...keys, ...args)</code>	Redisson does the same inside <code>RLock ; Jedis eval</code>	<code>redis.register_script</code> or <code>eval</code>
<code>(await redis.eval(...)) as [number, string]</code> : a tuple assertion	<code>(List<Object>) result</code> , then casts per element	tuple unpacking; no check
<code>z.string().regex(/^[a-z0-9][a-z0-9-]{0,31}\$/)</code> as a reusable field	a Bean Validation <code>@Pattern</code>	a pydantic <code>constr(pattern=...)</code>
eight separate <code>Redis</code> clients in one test, so the race is real	eight <code>JedisPool</code> connections	eight <code>redis.Redis()</code> instances
<code>redis.on("error", ...) logging err.message</code> only	a <code>Logger</code> with the exception's message, not <code>toString</code>	<code>logging.warning(str(err))</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The token issued only with the SET, or the join-race test could not count.

An earlier shape issued the token first (`INCR`) and then tried the `SET NX` , so ten racing joiners moved the counter by ten and the winner's token was whichever number its `INCR` happened to get. The test that asks "exactly one acquisition happened" cannot be written against that: token 1 for the winner is only meaningful if losers cost nothing. Moving the `INCR` after the `GET` check inside the one script made "the counter moved by one" a checkable fact, and the same script later took the floor (section 5) in the only place it can be applied safely: before the `INCR` , under the same atomicity.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The api acquiring leases on a node's behalf was rejected.

It looked simpler: `/session` knows the document and could pick a node and take the lease for it. But a lease has to be *renewed*, and only the process that holds the room knows whether it still holds the room. A node that did not acquire cannot know it must renew, and a lease renewed by a third party is a lease that outlives the room it protects. The node takes its own lease when its first peer arrives, and the api only reads.

The principle: a lease's three operations must each be atomic with their own check. Acquire checks "free" and takes; renew checks "still mine" and extends; release checks "still mine" and deletes. Redis gives atomicity for free inside one script, so the design puts each check and its effect in one script and never splits them across a round trip. Section 5 applies the same rule to Postgres, where the one statement is the atom.

2 · Clocks against tokens: when a holder must stop believing itself

A lease is a promise with a time on it, and a time raises the question every distributed-systems text spends a chapter on: *whose* time? Redis expires the key on its own clock, starting when it *receives* the command. The holder can only read its own clock. The two clocks run at slightly different rates, the message between them takes time, and the holder can be paused for a minute by the operating system, the garbage collector or `docker pause` without noticing. `keepLease` is the answer to "until when may this process act as the owner", and it is deliberately conservative.

```

export type LeaseKeeper = {
  /** False from the moment this process can no longer be sure it holds the lease. Never true again. */
  readonly held: boolean;
  /** Renew once (call every ttl/3). Also the moment an unrenovable lease is declared lost. */
  tick(): Promise<void>;
};

/**
 * Keeps a lease that was just acquired, and decides when to stop trusting it.
 *
 * Only OUR monotonic clock is used, never a wall clock, never a time from another machine: Redis starts a
 * PX countdown when it RECEIVES the command, which is after we sent it, so "sent + ttl" (on our own clock)
 * is never later than Redis's expiry. The margin covers the two clocks running at slightly different rates.
 * `renew` answers false when the lease is someone else's now (lost at once) and throws when Redis cannot be
 * asked (still ours until the deadline passes). `acquiredAt` must be read BEFORE the acquire was sent.
 *
 * This is not safety: a process frozen past the deadline believes `held` until it runs again. The journal's
 * fence (E7.3) is what refuses that zombie's writes; this only makes a live node give its peers up promptly.
 */
export function keepLease({ renew, ttlMs, now, acquiredAt, onLost }: { renew: () => Promise<boolean>; ttlMs:
number; now: () => number; acquiredAt: number; onLost: () => void }): LeaseKeeper {
  const trusted = (sentAt: number): number => sentAt + ttlMs * 0.9;
  let validUntil = trusted(acquiredAt);
  let lost = false;
  let renewing = false;
  const lose = (): void => {
    if (lost) return;
    lost = true;
    onLost();
  };
  const expired = (): boolean => {
    if (!lost && now() >= validUntil) lose();
    return lost;
  };
  return {
    get held() { return !expired(); },
    async tick() {
      // A renewal still in flight (a hung Redis, bounded by the client's command timeout) is not waited
      // for twice, but the deadline is still checked: a hang must not keep a room alive.
      if (expired() || renewing) return;
      renewing = true;
      const sentAt = now();
      try {
        if (await renew()) validUntil = trusted(sentAt);
        else lose();
      } catch {
        // Redis did not answer: keep trying on the next tick while the deadline allows.
      } finally {
        renewing = false;
      }
      expired();
    },
  };
}

```

Three decisions, each with a test. **Only our monotonic clock.** `now` is `performance.now()` in production, which cannot jump when NTP corrects the wall clock (`Date.now()` can, backwards). Java's `System.nanoTime()` and Python's `time.monotonic()` are the same instrument. **Counting from the send, not the reply.** Redis starts its `PX` countdown when the command arrives, which is after we sent it, so a deadline of `sent + ttl` on our clock can never be later than Redis's expiry; the `0.9` is the margin for the two clocks drifting. The test "the deadline counts from when a renewal was SENT" has a renewal that takes 800 ms to answer and asserts the lease is trusted until `300 + 900`, not `1100 + 900`. **A hang does**

not extend anything. A renewal in flight is not awaited twice, but the deadline is still checked on every `tick` and every read of `held`, so a hung Redis cannot keep a room alive by never answering.

The idiom to notice is that `held` is a getter with a side effect: reading it may fire `onLost`. That is a deliberate trade. The room's server needs "are we still the owner" to be true at the moment of the question, not at the last timer tick, and the cheapest way to make every reader honest is to make the read itself check the clock. `lose` is idempotent so that `onLost` runs once however many readers ask.

FROM `packages/lease/src/lease.test.ts`

```
it("the deadline counts from when a renewal was SENT, not when its answer came back", async () => {
  let clock = 0;
  const keeper = keepLease({
    renew: () => { clock += 800; return Promise.resolve(true); }, // a slow answer: Redis started its
    countdown at the send
    ttlMs: TTL, now: () => clock, acquiredAt: 0, onLost: () => undefined,
  });
  clock = 300;
  await keeper.tick(); // sent at 300, answered at 1100: trusted until 300 + 900
  clock = 1150;
  expect(keeper.held).toBe(true);
  clock = 1200;
  expect(keeper.held).toBe(false);
});

it("a renewal that hangs does not keep the lease: the deadline is checked even while one is in flight",
  async () => {
    let clock = 0;
    let lostCount = 0;
    const keeper = keepLease({ renew: () => new Promise<boolean>(() => undefined), ttlMs: TTL, now: () =>
    clock, acquiredAt: 0, onLost: () => { lostCount += 1; } });
    clock = 300;
    void keeper.tick();
    clock = 950;
    await keeper.tick();
    expect(lostCount).toBe(1);
    expect(keeper.held).toBe(false);
  });
```

Now the point the doc comment makes in its last paragraph, because it is the point of the epic. *This is not safety.* A process frozen for two seconds does not run `expired()` during those two seconds; when it wakes, its first read of `held` says false and it gives up, but between the wake and that read it may already have done something, and an `await` it was inside when it froze will simply continue. The learning test measured exactly that: owner A blocked its own event loop with `Atoms.wait` for two seconds, owner B took the lease 1.7 seconds before A woke, and A's in-memory flag still said "mine". Kleppmann's argument in *How to do distributed locking* (and chapter 8 of DDIA) is that no amount of care with clocks closes this gap, because the pause can happen at *any* instruction, including the one after the check. The only thing that closes it is a check by the storage, at write time, of a token that the frozen process cannot have updated. That is section 5; `keepLease` only makes a *live* node let go promptly, so that failover takes one ttl rather than a ttl plus a discovery.

FROM `learning-tests/redis/test.ts`

```
// 4. Zombie lease when the client's event loop is blocked past the TTL:
//   CONFIRMED, and re-verified with much wider slack and Redis-side (not
//   wall-clock) timestamps than an earlier draft of this test allowed for.
//   Owner A acquired a 300ms lease (fencing token 1 from the same Lua
//   acquire script as test 3). The main thread then synchronously blocked
//   its OWN event loop for 2000ms via Atoms.wait on a SharedArrayBuffer
//   (measured: unblocked after ~2001-2002ms). A separate OS process (owner
//   B) polled independently and, on winning the lease, recorded Redis's own
```

```
// `TIME` at the moment of acquisition (fencing token 2); A also recorded
// Redis's own `TIME` immediately after unblocking. Measured margin between
// B's Redis-side acquisition and A's Redis-side unblock: ~1695-1698ms
// across repeated runs -- B won the lease and moved on roughly 1.7 seconds
// before A even had a chance to notice anything was wrong, ruling out any
// "B only won after A unblocked" ambiguity. Right after unblocking, A's
// in-memory "I own it" flag was still `true` (measured/logged) while
// Redis's `GET lease` already returned `owner-B` (measured/logged) --
// A is a confirmed zombie. A's own CAS-renew Lua script (the same
// compare-and-set from test 2) was then run as owner-A and returned `0`
// (measured/logged): Redis's compare-and-set correctly refuses to extend
// a lease for a client that no longer holds it. The fencing token B
// acquired (2) was confirmed strictly greater than A's (1) (measured/
// logged). Implication: the lease/lock alone is not sufficient for safety
// under GC pauses, blocked event loops, or network partitions -- the
// *storage layer* must check the fencing token on every write and reject
// stale (lower) tokens, not just trust "I hold a lock" (a naive in-memory
// flag, as demonstrated here, survives being wrong indefinitely on its
// own).
//
```

Typescript (this repo)

Java

Python

`performance.now()`: monotonic, milliseconds, a double

`System.nanoTime()`

`time.monotonic()`

a getter with a side effect (`get held()` may fire `onLost`)

an `isHeld()` method that also fires a listener; unusual, and documented

a `@property` that calls a callback; unusual

`now: () => number` injected, wound by hand in tests

an injected `Clock` (`java.time.Clock`)

an injected callable; `freezegun` for wall clocks only

`Atomics.wait` on a `SharedArrayBuffer` to block the loop on purpose

`Thread.sleep` on the one thread that matters

`time.sleep`

renewing as a plain boolean guard

an `AtomicBoolean`

a bool; the GIL and the loop make it safe

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Trusting the reply time would have given a slow Redis a longer lease than it granted.

The obvious deadline is "when the renewal came back plus ttl". With an 800 ms round trip that trusts the lease for 800 ms longer than Redis will keep it: another node could acquire while this one still believes and still accepts ops. Every op accepted in that window is one the fence must refuse later, and one whose sender was told nothing for the whole window. Reading the clock before the send, into `sentAt`, and passing it to `trusted` after the answer is the whole fix; the test with the slow renewal is the mutation check.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A renewal that fails must not shorten what an earlier one earned.

A first version set `validUntil` from the failed attempt's send time when Redis threw, which under a flaky network would have made the room give up its peers every time one renewal failed, although the previous one had bought it another 900 ms. The rule became: a refusal (`false`) loses at once, a throw changes nothing and is tried again next tick, and only a `true` moves the deadline. The test winds the clock to 1350 after a failed renewal at 800 and asserts still held.

Principal-level point: there are two independent instruments here and they answer different questions. The *clock* answers "may I still act as the owner" and it is allowed to be wrong in one direction only, towards giving up early. The *token* answers "was this write made by the current owner" and it is never wrong, because it is compared by the storage, not by the process that might be frozen. A system that uses only the first is unsafe under pauses; a system that uses only the second is safe but has no failover time bound, because nobody ever gives anything up. This repo uses both, and keeps them in separate modules so that each has its own tests.

3 · One room globally: the loser says 4409, and /session knows the owner

The lease decides who owns a room; the rest of F20 is making every peer arrive at the owner without the peer knowing anything about nodes. Lesson 2 already had the hook: a browser never dials a sync address it remembers, it asks `POST /documents/:id/session` and dials what comes back. So the routing lives in one place, and it needs one input: a table of nodes.

FROM `packages/lease/src/lease.ts`

```
/**
 * The sync nodes a peer can be sent to, from one variable: a single `ws(s)://` URL (one node, no lease
 * lookup: the single-node setup and the tests), or `id=url,id=url` (the routing table of epic 7).
 */
export type SyncNodes = { kind: "one"; url: string } | { kind: "many"; nodes: ReadonlyMap<string, string> };

// A query or fragment would swallow the "/documents/<id>" that gets appended to the address.
const wsBase = (value: string): string | undefined => {
  if (!URL.canParse(value)) return undefined;
  const url = new URL(value);
  return ["ws:", "wss:"].includes(url.protocol) && url.search === "" && url.hash === "" &&
    !value.includes("?") && !value.includes("#") ? value.replace(/\./+$/, ".") : undefined;
};

export function parseSyncNodes(value: string): SyncNodes | undefined {
  if (!value.includes("=")) {
    const url = wsBase(value);
    return url === undefined ? undefined : { kind: "one", url };
  }
  const nodes = new Map<string, string>();
  for (const entry of value.split(",").map((part) => part.trim()).filter(Boolean)) {
    const at = entry.indexOf("=");
    const id = entry.slice(0, at).trim();
    const url = wsBase(entry.slice(at + 1).trim());
    if (!NodeId.safeParse(id).success || url === undefined || nodes.has(id)) return undefined;
    nodes.set(id, url);
  }
  return nodes.size > 0 ? { kind: "many", nodes } : undefined;
}
```

`SyncNodes` is a discriminated union with two shapes, and the `"one"` shape exists so that nothing changed for the single-node setup or for any test: one URL, no lookup, no Redis asked. `parseSyncNodes` is strict about what would route a peer nowhere: a query or fragment (it would swallow the `/documents/<id>` appended later), an `http://` scheme, a duplicate id, an id `NodeId` refuses. `URL.canParse` is Node 24's way to ask "is this a URL" without a `try`. The same field is used by the api (`SYNC_PUBLIC_URL`, what browsers dial) and by the worker (`SYNC_URL`, addresses inside the Docker network), because the two audiences reach the nodes through different networks.

FROM `apps/api/src/config.ts`

```
// The address BROWSERS use to reach the sync server (not the address inside the Docker network). One URL
for
// one node; with several (E7.1) the routing table `id=url,id=url`, ids as the nodes' SYNC_NODE_ID.
SYNC_PUBLIC_URL: syncNodesField("SYNC_PUBLIC_URL"),
```

FROM `apps/worker/src/config.ts`

```
// How THIS process reaches the sync server. Not SYNC_PUBLIC_URL: that one is for browsers. One URL, or with
// several nodes (E7.1) the routing table `id=url,id=url` over their addresses inside the Docker network.
SYNC_URL: syncNodesField("SYNC_URL"),
```

FROM `packages/lease/src/lease.ts`

```
/**
 * Where a peer of `documentId` dials: the room's owner if it has one and is alive, otherwise any live node
 * (`pick`), whose first peer then tries to take the lease. Two nodes racing for a free room is fine: exactly
 * one
 * wins, and the other closes its peers with 4409, sending them back here, where the owner is now known.
 * An owner that stopped beating (killed, E7.2) still holds its lease until it expires: its peers are sent to
 * a
 * live node, which waits that lease out and takes the room (takeLease below), instead of dialling a dead one.
 * `owner` or `alive` throwing means Redis cannot be asked: the caller answers "try again".
 * Liveness is only a hint for routing: a node wrongly thought dead costs a peer one 4409, never a second
 * room.
 */
export function syncRouter({ nodes, owner, alive, pick = randomNode }: { nodes: SyncNodes; owner: (documentId:
string) => Promise<Holder | undefined>; alive: (nodeIds: readonly string[]) => Promise<ReadonlySet<string>>;
pick?: (ids: readonly string[]) => string }): (documentId: string) => Promise<string> {
  return async (documentId) => {
    if (nodes.kind === "one") return `${nodes.url}/documents/${documentId}`;
    const ids = [...nodes.nodes.keys()];
    const [holder, beating] = await Promise.all([owner(documentId), alive(ids)]);
    // Owned by a node this table does not name: sending the peer anywhere else would loop on 4409 for ever.
    if (holder && !nodes.nodes.has(holder.nodeId)) throw new Error(`the room is owned by sync node
"${holder.nodeId}", which is not in the routing table`);
    const live = ids.filter((id) => beating.has(id));
    // Nobody beating at all (Redis just restarted, or heartbeats cannot be written): route as if all were
    alive.
    const id = holder && (beating.has(holder.nodeId) || live.length === 0) ? holder.nodeId : pick(live.length
> 0 ? live : ids);
    return `${nodes.nodes.get(id) ?? ""}/documents/${documentId}`;
  };
}
```

The router's contract, in order. An owner that beats: its address. An owner that stopped beating: any live node, which will wait the dead lease out (section 4). No owner: any live node, whose first peer takes the lease. Nobody beating at all: route as if everyone were alive, because "no heartbeats" after a Redis restart must not mean "no service". An owner the table does not name: an error, because sending the peer anywhere else would bounce on 4409 for ever, and a loud error is how a misconfigured table is found. And Redis unreachable propagates, so the caller can say "try again" honestly. `pick` is injectable so the tests can force a first dial to the wrong node, which is how the race in F20 is provoked on purpose.

FROM apps/api/src/app.ts

```
// The routing hook (SPEC §2.11): a peer never knows a sync address in advance, it asks here, and gets the
// node that owns the room (E7.1), or any node when nobody does yet. A viewer opens the document too: to
watch
// it. What they may do inside is the sync server's business, which reads the role itself (the token says
who,
// never what they may do, so a role change is not waiting for a token to expire).
document.post("/session", need("viewer"), async (c) => {
  const { doc } = c.var;
  // E5.3a: a document is opening, so the git peer looks at Gitea now: a push whose delivery was lost
reaches
  // the canvas at once, not on the next timer. Best effort: a session is never refused over it.
  await db.gitStore().requestReconcile().catch((err: unknown) => {
    process.stderr.write(`${JSON.stringify({ level: "warn", path: c.req.path, message: `reconcile not
requested: ${describeError(err)}\n`});
  });
  let wsUrl;
  try {
    wsUrl = await route(doc.id);
  } catch (err) {
    // Redis cannot say who owns the room, and no node would open it without knowing: "try again".
    process.stderr.write(`${JSON.stringify({ level: "warn", path: c.req.path, message: `no sync node:
${describeError(err)}\n`});
    return fail(c, 503, "sync_unavailable");
  }
  const now = Math.floor(Date.now() / 1000);
  const token = signSessionToken({ userId: c.var.user.id, name: c.var.user.name, orgId: doc.orgId,
documentId: doc.id, secret: sessions.secret, ttlSeconds: sessions.ttlSeconds, now });
  return c.json({
    wsUrl,
    token,
    expiresAt: new Date((now + sessions.ttlSeconds) * 1000).toISOString(),
    // E10.8: the role the middleware just read, beside the token and never in it: the editor shows Share to
an owner only.
    role: c.var.role,
  } satisfies SessionResponse);
});
```

FROM apps/api/src/main.ts

```
// Several sync nodes: /session reads each room's owner from its lease. One node: nothing to ask.
const leases = config.sessions.sync.kind === "many" ? createLeases({ redisUrl: config.redisUrl }) : undefined;
// E8.2 (F24): role changes are announced to the sync nodes, which apply them to open sessions.
const access = accessPublisher({ redisUrl: config.redisUrl });
```

FROM apps/worker/src/live.ts

```
/**
 * How THIS process reaches a document's room (not the browsers' address): one sync node's `syncUrl`, or, with
 * several nodes (E7.1), a `route` to whichever node owns the room. Asked before every connection, like
/session.
 */
export type SyncSessions = { secret: string } & ({ syncUrl: string } | { route: (documentId: string) =>
Promise<string> });

export const roomUrl = (sessions: SyncSessions, documentId: string): Promise<string> =>
  "route" in sessions ? sessions.route(documentId) :
  Promise.resolve(`${sessions.syncUrl}/documents/${documentId}`);

/** SYNC_URL (config.ts) made usable: a table of nodes reads each room's owner from its lease in Redis. */
export function syncSessions(config: { secret: string } & ({ syncUrl: string } | { nodes: ReadonlyMap<string,
string> }), redisUrl: string): { sessions: SyncSessions; close: () => Promise<void> } {
  if (!("nodes" in config)) return { sessions: config, close: () => Promise.resolve() };
  const leases = createLeases({ redisUrl });
```

```

    return { sessions: { secret: config.secret, route: syncRouter({ nodes: { kind: "many", nodes: config.nodes
}, owner: (documentId) => leases.owner(documentId), alive: (nodeIds) => leases.alive(nodeIds) }) }, close: ()
=> leases.close() };
}

```

The worker's AI, preview, ship and git peers used to dial `ws://sync:3001` directly. With two nodes that would have bounced on 4409 for every room the other node owns, for ever, because those peers had no `/session` to ask.

`syncSessions` gives them the same router the api has, asked before every connection. It is a seam like `enqueue`: the app takes `owner` and `alive` as functions, so the api's tests run with no Redis at all (the default rejects, and the single-node shape never calls it).

Now the node's side. A room opens only under the lease. `open()` takes it, starts renewing at once, loads the room, and hands back a room that also knows its sockets, so that losing the lease can send every peer elsewhere.

FROM `apps/sync/src/server.ts`

```

// 4000-4999 are ours to define. They mirror the HTTP status a REST call would have had.
// roomElsewhere (a 409, "conflict"): another sync node owns this document's room. Not fatal to a peer: it
// asks /session again, which answers with the owner's address.
const CLOSE = { invalidMessage: 4400, documentNotFound: 4404, roomElsewhere: 4409, tooManyRequests: 4429,
documentCorrupt: 4500, unavailable: 4503 } as const;

```

FROM `apps/sync/src/server.ts`

```

/**
 * Room ownership across sync nodes (F20): a room opens here only while this node holds the document's lease
 * in Redis. Without it this process opens every room it is asked for, which is right for exactly one node.
 */
lease?: { leases: Leases; nodeId: string };

```

FROM `apps/sync/src/server.ts`

```

/**
 * Takes the document's lease (when there are several nodes), then loads the room. Never rejects.
 * `forget` drops this room from `rooms` if it is still the current one there.
 */
async function open(documentId: string, orgId: string, forget: () => void): Promise<Opened> {
    const owned = (loaded: Loaded, release: () => Promise<void>): Owned => ({ ...loaded, sockets: new Set(),
lost: false, release });
    if (!lease) {
        const loaded = await load(documentId, orgId);
        return "closeCode" in loaded ? loaded : owned(loaded, () => Promise.resolve());
    }
    const { leases, nodeId } = lease;
    // The largest token that ever claimed the document (E7.3): ours must be above it, even after a Redis
flush.
    let floor = 0;
    if (store) {
        let fence;
        try {
            fence = await store.fence(orgId, documentId);
        } catch {
            return { closeCode: CLOSE.unavailable };
        }
        if (fence === undefined) return { closeCode: CLOSE.documentNotFound };
        floor = fence;
    }
    let holder: Holder;
    let acquiredAt: number; // BEFORE the winning send: our deadline must never outlast Redis's (lease.ts)
    try {
        // Another live node owns the room: its peers are sent back to /session, which names the owner. A dead
// one's lease (or our own previous run's) is waited out here, up to one ttl, and then the room is ours

```

```

(F21).
    const taken = await takeLease({
      acquire: () => leases.acquire(documentId, nodeId, floor), alive: async (id) => (await
leases.alive([id])).has(id),
      nodeId, ttlMs: leases.ttlMs, now: () => performance.now(), sleep: (ms) => new Promise((resolve) =>
setTimeout(resolve, ms).unref()),
    });
    if (!taken) return { closeCode: CLOSE.roomElsewhere };
    ({ holder, acquiredAt } = taken);
  } catch (err) {
    // Redis cannot say who owns it: opening anyway could make a second room. "Try again" instead.
    log(documentId, `lease not taken: ${err instanceof Error ? err.message : "unknown"}`);
    return { closeCode: CLOSE.unavailable };
  }
  // Renewal starts NOW, not after the load: a long replay must not let the lease lapse unnoticed.
  const opened: { room?: Owned } = {}; // filled once loaded
  // The lease ran out (our clock) or the journal fenced us (a newer owner claimed the document): stop
writing,
  // send the peers to the new owner.
  const drop = (why: string): void => {
    clearInterval(renewing);
    const { room } = opened;
    if (!room || room.lost) return; // still loading: the check after load() below refuses it
    log(documentId, `lease ${String(holder.token)} ${why}: its peers are sent to the new owner`);
    room.lost = true;
    room.snapshot?.stop();
    forget(); // the next peer asks Redis again
    for (const socket of room.sockets) socket.close(CLOSE.roomElsewhere, "room_elsewhere");
  };
  const keeper = keepLease({
    renew: () => leases.renew(documentId, holder), ttlMs: leases.ttlMs, now: () => performance.now(),
    acquiredAt,
    onLost: () => { drop("lost"); },
  });
  const renewing = setInterval(() => void keeper.tick(), leases.ttlMs / 3);
  renewing.unref();
  const release = async (): Promise<void> => {
    clearInterval(renewing);
    if (!keeper.held) return;
    await leases.release(documentId, holder).catch((err: unknown) => {
      log(documentId, `lease not released, it expires by itself: ${err instanceof Error ? err.message :
"unknown"}`);
    });
  };
}

```

Read `drop` and `keepLease`'s `onLost` together: when the deadline passes or a renewal is refused, the room is marked `lost`, snapshots stop, the room is forgotten (so the next peer asks Redis again rather than joining a corpse), and every socket is closed with 4409. Renewal starts *before* `load()`, because replaying a long journal can take longer than a third of a `ttl`. And a room whose lease lapsed *during* the load is refused at the end (`!keeper.held`), released, and answered 4409, so a peer never joins a room that is already dead.

FROM `apps/sync/src/server.ts`

```

if (opened.lost) {
  ws.close(CLOSE.roomElsewhere, "room_elsewhere"); // it lost its lease while this peer was on its way in
  return;
}

```

FROM `apps/sync/src/server.ts`

```

if (opened.lost) return; // being sent elsewhere: take nothing more (an op already in the queue meets
the fence)

```

4409 is a *conflict* close code by analogy with HTTP 409, and it is the one code in the 4000s that is not fatal to a peer. peer-client's `FATAL_CLOSE_CODES` does not include it, so the peer does what it does on any non-fatal close: asks `session()` again and dials what it answers, which by now names the owner. Lesson 2's client needed no change for F20 except one test pinning this.

FROM `packages/peer-client/src/peer.ts`

```
const FATAL_CLOSE_CODES = new Set([4400, 4404, 4500, 1009]);
```

FROM `packages/peer-client/src/peer.test.ts`

```
test("the room's node dies, or says the room moved (4409): the next connection dials whatever /session answers NOW (F21)", async () => {
  const net = fakeNet((socket, nth) => {
    socket.say(welcome);
    if (nth === 1) socket.close(1006); // killed
    if (nth === 2) socket.close(4409); // not the owner (yet)
  });
  const addresses = ["ws://node-a/documents/d", "ws://node-b/documents/d", "ws://node-c/documents/d"];
  let asked = 0;
  const peer = connectPeer(options(net, { session: () => Promise.resolve({ wsUrl: addresses[Math.min(asked++, 2)] ?? "", token: "t" }) }));
  await until(() => net.sockets.length === 3 && peer.status === "live", "live on the third address");
  expect(net.sockets.map((socket) => socket.url)).toEqual(addresses);
  peer.close();
});
```

FROM `docker-compose.yml`

```
# One room per open document, on exactly ONE of the sync nodes: whichever holds the document's lease in
# Redis (E7.1, F20). Two nodes, 'sync' and 'sync-2'; the service name is the node id the routing tables use.
sync: &sync
  build: .
  command: ["node", "apps/sync/src/main.ts"]
  environment: &sync-env
  PORT: "3001"
  SYNC_NODE_ID: sync
  # Through redis-proxy, on this node's own listener (E7.3); the api and the workers dial Redis directly.
  REDIS_URL: "redis://:${REDIS_PASSWORD}?run ./init.sh@redis-proxy:6379"
  # How long a room outlives a silent owner (tunable; empty = 10 s). Renewed every third of it.
  LEASE_TTL_MS: "${LEASE_TTL_MS:-}"
  DATABASE_URL: "postgres://noon_app:${APP_DB_PASSWORD}?run ./init.sh@postgres:5432/noon"
  SESSION_TOKEN_SECRET: "${SESSION_TOKEN_SECRET:?run ./init.sh}"
  MINIO_URL: http://minio:9000
  MINIO_USER: noon
  MINIO_PASSWORD: "${MINIO_PASSWORD:?run ./init.sh}"
  # Tunable from the shell (SPEC §2.9); empty = the defaults, 500 ops and 30 s.
  SNAPSHOT_EVERY_OPS: "${SNAPSHOT_EVERY_OPS:-}"
  SNAPSHOT_EVERY_SECONDS: "${SNAPSHOT_EVERY_SECONDS:-}"
  ports: ["127.0.0.1:${SYNC_PORT:-3001}:3001"]
  depends_on:
    migrate: { condition: service_completed_successfully }
    minio: { condition: service_healthy }
    redis-proxy: { condition: service_healthy }
  healthcheck:
    test: ["CMD", "node", "-e", "fetch('http://localhost:3001/health').then(r => process.exit(r.ok ? 0 : 1), () => process.exit(1))"]
    interval: 2s
    timeout: 2s
    retries: 30
  # The second node: the same service, another id and host port (3002 is Gitea's).
  sync-2:
    <<: *sync
```

```
environment:
  <<: *sync-env
  SYNC_NODE_ID: sync-2
  REDIS_URL: "redis://:${REDIS_PASSWORD}?run ./init.sh}@redis-proxy:6380"
ports: ["127.0.0.1:${SYNC_2_PORT:-3003}:3001"]
```

FROM `docker-compose.yml`

```
# What a BROWSER on this machine dials to reach each sync node: id=url per node, ids as their
SYNC_NODE_ID
# (E7.1). /session answers with the address of the node that owns the document's room.
# Hosting through one public host (Vite proxies /sync and /sync-2 to the two nodes), set in the shell,
not .env:
#   SYNC_PUBLIC_URL=sync=wss://<public host>/sync,sync-2=wss://<public host>/sync-2
  SYNC_PUBLIC_URL: "${SYNC_PUBLIC_URL:-sync=ws://localhost:${SYNC_PORT:-3001},sync-
2=ws://localhost:${SYNC_2_PORT:-3003}}"
```

FROM `apps/web/vite.config.ts`

```
// One path per sync node (E7.1): the api's SYNC_PUBLIC_URL=sync=wss://<host>/sync,sync-
2=wss://<host>/sync-2.
// Anchored patterns: a plain "/sync" prefix would also swallow "/sync-2".
"^/sync(/|$)": { target: "ws://localhost:3001", ws: true, rewrite: (path) => path.replace(/^\/sync/, "")
},
  "^/sync-2(/|$)": { target: "ws://localhost:3003", ws: true, rewrite: (path) => path.replace(/^\/sync-2/,
  "") } ,
```

FROM `apps/sync/src/one-room-globally.int.test.ts`

```
test("a node whose lease is taken sends its peers away with 4409, and they land in the new owner's room",
  async () => {
    const documentId = randomUUID();
    const peers = [peer(documentId, "node-a"), peer(documentId, "node-a")];
    await until(() => peers.every((each) => each.status === "live"), "both peers live on node-a", 5000);
    expect(nodes.roomAt(documentId)).toBe(0);
    const before = await nodes.owner(documentId);

    // What a long pause of node-a looks like from outside: its lease is gone, and node-b takes the room.
    await nodes.leases.release(documentId, before ?? { token: 0, nodeId: "none" });
    const usurper = await connect(nodes.servers[1].url, documentId);
    await usurper.next("welcome");
    const after = await nodes.owner(documentId);
    expect(after).toMatchObject({ nodeId: "node-b" });
    expect(after?.token).toBeGreaterThan(before?.token ?? Infinity); // a later owner always carries a larger
    token

    // node-a's next renewal (every ttl/3) is refused: it gives its peers up; they ask again and join node-b.
    await until(() => nodes.servers[0].peerCount(documentId) === 0 && nodes.servers[1].peerCount(documentId) ===
    3, "every peer on node-b", 5000);
    await until(() => peers.every((each) => each.status === "live"), "both peers live again", 5000);
    expect(nodes.roomAt(documentId)).toBe(1);
    for (const each of peers) each.close();
    usurper.close();
  });
```

FROM `apps/sync/src/one-room-join-race.int.test.ts`

```
// integration:one-room-join-race (E7.1, F20). N peers of a brand-new document join at the same moment, split
// across both sync nodes before anyone owns the room: both nodes race for the lease, exactly one wins, and it
// issues exactly one fencing token; the loser sends its peers back to /session, and all N end in ONE room.
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>type SyncNodes = { kind: "one"; url } { kind: "many"; nodes: ReadonlyMap }</code>	a sealed interface with two records	a <code>Union</code> of two dataclasses with a <code>Literal</code> tag
<code>URL.canParse(value)</code>	<code>new URI(value)</code> in a <code>try</code>	<code>urllib.parse.urlparse</code> , which never throws, so check the parts
<code>owner = () => Promise.reject(new Error("no lease store"))</code> as a default that refuses	a constructor overload that throws <code>UnsupportedOperationException</code>	a default argument that raises
YAML anchors: <code>sync: &sync</code> , <code>sync-2: { <<: *sync }</code>	the same YAML	the same YAML
a regex key in Vite's proxy: <code>"^/sync(/ \$)"</code>	a Spring Cloud Gateway route with an anchored predicate	an nginx <code>location ~ ^/sync(/ \$)</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A plain `/sync` proxy prefix swallowed `/sync-2`.

The public demo runs through one host, and Vite forwards `/sync` to node one and `/sync-2` to node two. With prefix keys, every `/sync-2/documents/...` request matched `/sync` first and was rewritten to `-2/documents/...` on the wrong node. Anchoring the patterns (`^/sync(/|$)`) is a one-line fix, and the comment says why so nobody "simplifies" it back. A single-URL `SYNC_PUBLIC_URL` against two nodes is the same class of misconfiguration: browsers bounce on 4409 for every room the other node owns, and the commit names it so an operator recognises the symptom.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Hashing the document id to pick a node was rejected.

Consistent hashing is the textbook answer for "which node", and it is wrong here: a dead node would be picked every time for its share of documents, and a peer's only remedy would be to keep retrying until the lease moved. The lease is already the truth about ownership; the pick only matters for a free room, and there it should prefer a node that is alive. Section 4's heartbeats are that preference.

The shape: ownership is decided in one place (the lease), advertised through one door (`/session`), and enforced by the owner refusing what is not its (4409). A peer never needs a routing table, a node never needs to know the other nodes, and the api never needs to know which rooms are open. Each of them asks Redis one question and acts on the answer.

4 · Failover: heartbeats as a hint, and a dead lease waited out

E7.1 left a named gap: kill the owner and its lease pins its peers to a dead address until it expires, and afterwards a random pick sends half of them back to the corpse. With peer-client's backoff (up to ten seconds) the room could come back a second ttl late. F21 budgets one ttl. Two additions close the gap, and both are careful to add *speed* without adding a second source of truth about ownership.

FROM packages/lease/src/index.ts

```
async beat(nodeId) {
  await redis.set(`${prefix}node:${nodeId}`, "1", "PX", Math.max(1, Math.round(ttlMs / 3)));
},
async alive(nodeIds) {
  if (nodeIds.length === 0) return new Set();
  const beats = await redis.mget(...nodeIds.map((id) => `${prefix}node:${id}`));
  return new Set(nodeIds.filter((_, i) => beats[i] !== null));
},
```

FROM apps/sync/src/server.ts

```
// "Alive" into Redis every tenth of a ttl (E7.2): /session stops sending peers to a node that went quiet,
// and a node knows which leases can only expire. A failed beat is not logged: Redis being away already is.
const beat = (): void => { if (lease && !closing) void lease.leases.beat(lease.nodeId).catch(() =>
undefined); };
const beating = lease ? setInterval(beat, lease.leases.ttlMs / 10) : undefined;
beating?.unref();
beat();
```

Heartbeats. Every node writes `node:<id>` with an expiry of a third of a ttl, every tenth of one. A killed node is known dead within $\text{ttl}/3$, long before any of its leases expire. Two things are not done with that knowledge, on purpose. It does not decide ownership: the router uses it only to choose where a peer is *sent*, and a node wrongly thought dead costs a peer one 4409, never a second room. And it does not steal: a lease held by a node that stopped beating is still that node's until Redis lets it go, because "stopped beating" and "stopped writing" are different facts (a paused node does neither, and wakes).

FROM packages/lease/src/lease.ts

```
/**
 * A node's first peer of a room has arrived: take the lease. Held by a live node: refused at once (its peers
 * go to the owner). Held by a node that stopped beating, or under OUR id by a run of this node that is gone
 * (killed and restarted): that lease can only expire, so wait for it, polling, for at most one ttl, then take
 * the room (F21). Never stolen early: until it expires, the dead holder's lease is the only truth there is.
 * `acquiredAt` is read BEFORE the winning acquire was sent (keepLease's deadline counts from it).
 * ponytail: polls every ttl/10 rather than waiting for the key's exact expiry; ceiling: the takeover lands up
 * to
 * ttl/10 after the lease expired. Upgrade: return the PTTL from the acquire script and sleep exactly that.
 */
export async function takeLease({ acquire, alive, nodeId, ttlMs, now, sleep }: { acquire: () => Promise<{
acquired: boolean; holder: Holder }>; alive: (nodeId: string) => Promise<boolean>; nodeId: string; ttlMs:
number; now: () => number; sleep: (ms: number) => Promise<void> }): Promise<{ holder: Holder; acquiredAt:
number } | undefined> {
  const giveUpAt = now() + ttlMs + ttlMs / 10;
  for (;;) {
    const acquiredAt = now();
    const taken = await acquire();
    if (taken.acquired) return { holder: taken.holder, acquiredAt };
    if (taken.holder.nodeId !== nodeId && (await alive(taken.holder.nodeId))) return undefined;
    if (now() >= giveUpAt) return undefined; // it did not expire when it should have: renewed after all
    await sleep(ttlMs / 10);
  }
}
```

Waiting the lease out. `takeLease` is the pure decision a node makes when its first peer of a room arrives and the acquire is refused. Held by a live node: refuse at once, the peer belongs with the owner. Held by a node that stopped beating, or by *our own id* from a previous run that was killed: poll every $\text{ttl}/10$ until the lease expires, then take it; if it does not expire within about a ttl it was renewed after all, so refuse. The peer's socket stays open the whole time,

connecting, so the takeover lands within $\text{ttl}/10$ of the expiry rather than at the peer's next backoff. This is also what retires E7.1's ponytail: a restarted node no longer bounces its own peers with 4409 for up to a ttl.

The `for (;;)` with a return in the middle is the idiom for a poll with a deadline; `acquiredAt` is read before each acquire is sent, for the reason section 2 gave. The `sleep` is injected and the tests wind a fake clock through it, so a test of "waited out, taken within a poll of expiry" runs in microseconds and asserts on exact numbers.

FROM `packages/lease/src/lease.test.ts`

```
it("held by a node that stopped beating (killed): waited out, and taken no later than a poll after it expires", async () => {
  const { take, clock } = setup({ holder: { token: 4, nodeId: "sync-2" }, expiresAt: 700, beating: new Set(["sync"]) });
  const taken = await take();
  expect(taken?.holder).toEqual({ token: 5, nodeId: "sync" });
  expect(clock()).toBeLessThanOrEqual(700 + TTL / 10 + 1);
  expect(taken?.acquiredAt).toBeGreaterThanOrEqual(700 - TTL / 10);
});

it("our OWN id on it (this node was killed and restarted) is waited out too, though our id is beating", async () => {
  const { take } = setup({ holder: { token: 4, nodeId: "sync" }, expiresAt: 300, beating: new Set(["sync"]) });
  expect((await take)?.holder.token).toBe(5);
});
```

FROM `apps/sync/src/failover.int.test.ts`

```
// integration:failover (E7.2, F21). Two sync nodes, one Redis. The node that owns a room dies without a word
// (its Redis connection first, so it neither releases its lease nor beats again): its peers ask the router
// again,
// are sent to the live node, and that node takes the room as soon as the dead lease expires, never later than
// about one ttl after the kill. The data half of F21 (no acknowledged op lost or doubled) needs the journal:
// scripts/chaos/kill-owner-failover.ts, against the compose stack.
```

FROM `apps/sync/src/failover.int.test.ts`

```
test("killing the owner moves the room: its peers reach the live node through the router, within about one ttl", async () => {
  const documentId = randomUUID();
  const peers = [peer(documentId), peer(documentId)];
  await until(() => peers.every((each) => each.status === "live"), "both peers live on node-a", 5000);
  const before = await (lookup as Leases).owner(documentId);
  expect(before).toMatchObject({ nodeId: "node-a" });

  // kill: Redis first, so the lease is neither renewed nor released and the heartbeat stops; then the
  // sockets.
  killed = true;
  await (leases[0] as Leases).close();
  const killedAt = performance.now();
  await (servers[0] as RunningSyncServer).close();

  await until(() => peers.every((each) => each.status === "live") && (servers[1] as RunningSyncServer).peerCount(documentId) === 2, "both peers live on node-b", 5000);
  expect(performance.now() - killedAt).toBeLessThan(TTL + TTL / 10 + 500);
  const after = await (lookup as Leases).owner(documentId);
  expect(after).toMatchObject({ nodeId: "node-b" });
  expect(after?.token).toBeGreaterThan(before?.token ?? Infinity);
  for (const each of peers) each.close();
});
```

The integration test kills node-a the way a process dies: its Redis client is closed *first*, so the lease is neither renewed nor released and the heartbeat stops, then the sockets are torn down. Anything gentler (a clean `close()` releases the lease) would test a different, easier feature. The data half of F21 needs the journal and the real stack, and that is the chaos run.

FROM `scripts/chaos/kill-owner-failover.ts`

```
// chaos:kill-owner-failover (E7.2, SPEC F21). Against the REAL compose stack (two sync nodes):
// ./init.sh (or: docker compose up -d), then node scripts/chaos/kill-owner-failover.ts (ROUNDS=3 by
// default)
// Each round, on a fresh document: a person (the api's session route) and the AI (a token signed as its
// worker
// signs it, routed by /session too) join through @noon/peer-client and get a few edits acknowledged. Each
// fires a
// burst and the sync node that owns the room (read from its lease) is `kill -9`ed under it, as in E6.3. More
// edits
// are made while it is gone. It is NOT started again: the peers must find the OTHER node through /session by
// themselves, and that node must take the room as soon as the dead lease expires (never more than about one
// LEASE_TTL_MS after the kill). Then scripts/chaos/no-loss.ts, unchanged, holds the ledger against the
// journal
// (F18: every acknowledged op present, none twice, unacknowledged ops resent and applied once, peers
// converge).
// Pitfalls (SPEC §4a): the fault opens while the workload is on the wire; waits poll the observable; it
// repeats;
// the killed node is started again in `finally`. Prints one JSON line; exit 0 = PASS. The org is deleted at
// the end.
```

FROM `scripts/chaos/kill-owner-failover.ts`

```
const before = holderOf(doc);
if (!before) throw new Error(`no sync node holds the lease of ${doc}`);
victim = before.node;
edit(20); // on the wire...
compose("kill", "--signal", "SIGKILL", victim); // ...and the owner dies under it. execFileSync: no ack is
heard meanwhile.
const killedAt = Date.now();
ledger.fault();
await until(() => all.every([, p]) => p.status !== "live"), "both peers see the owner gone", 30_000);
edit(5); // made while no node has the room: held, sent after the next welcome

let after: ReturnType<typeof holderOf>;
await until(() => { after = holderOf(doc); return after !== undefined && after.node !== victim; },
`another node takes the room (budget ${String(takeoverBudgetMs)} ms)`, takeoverBudgetMs + 30_000);
const movedAfterMs = Date.now() - killedAt;
if (!after || after.token <= before.token) throw new Error(`the new owner's token ${String(after?.token)}
is not above the dead one's ${String(before.token)}`);
await until(() => all.every([, p]) => p.status === "live" && p.pendingCount === 0), "both peers
reconnected through /session, nothing pending", 30_000);
await ledger.settle(15_000);
await until(() => all.every([, p]) => p.seq === peers.person.seq), "both peers at the same seq", 10_000);
```

Everything lesson 6 said about the ledger applies unchanged, which was the point of writing `noLossViolations` as a pure function: a different fault, the same assertion. Two things are new. The victim is read from the lease (`holderOf`, `redis-cli GET lease:<id>`), because the run does not know which node the api routed the room to, and must kill *that* one. And the victim is *not* restarted until the round is over: the peers have to find the other node through `/session` by themselves, and the takeover time is measured against `t1 + t1/10 + 2 s`, so a takeover that waited out a second `t1` fails the run.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>redis.set(key, "1", "PX", ms)</code> as a heartbeat	Redisson's lock watchdog, or an etcd lease keep-alive	<code>r.set(key, 1, px=ms)</code>
<code>for (;;) { ... return ...; await sleep(ms); }</code> with a deadline	a <code>while (true)</code> with <code>Thread.sleep</code> ; Awaitility in tests	<code>while True:</code> with <code>await asyncio.sleep</code>
<code>sleep: (ms) => Promise<void></code> injected, advancing a fake clock	an injected <code>Sleeper</code>	an injected coroutine
<code>await leases.close()</code> before <code>server.close()</code> : an unclean death, on purpose	close the Jedis pool first	close the Redis client first
<code>Date.now() - killedAt</code> against a budget computed from the ttl	<code>Duration.between</code>	<code>time.time()</code> arithmetic

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Stealing the lease as soon as the heartbeat is gone was rejected.

It would make failover take $\text{ttl}/3$ instead of a ttl . It would also make a paused owner's writes overlap with the new owner's from the moment of the steal, with nothing yet to stop them (the fence is E7.3, built after). Even with the fence, a steal turns "the old owner is fenced when it wakes" from a rare event into the normal one, and every op it accepted meanwhile is an op its peers were told about and the journal refused. Waiting for the expiry keeps the invariant that at most one node *believes* it owns a room at any wall-clock moment, except during a pause, and pauses are what the fence is for.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Shortening peer-client's backoff was rejected.

The second-ttl delay came from the client's retry curve: a peer that bounced on 4409 backed off, and by the time it asked again the lease had long expired. The tempting fix is a faster curve. The chosen one is that the *server* holds the connecting peer while it waits the lease out, so the bound on takeover comes from the node, and every client, including the worker's peers, gets it without touching their retry behaviour.

CAP, honestly: during the ttl after a kill the room is *unavailable*: the old owner is gone, the new one refuses to take the room until the lease expires, and peers hold their edits. That is a consistency-over-availability choice, made because two rooms for one document is a divergence the design cannot repair (lesson 2), while a ten-second pause is something peer-client already absorbs. The dial is `LEASE_TTL_MS`: shorter means faster failover and more false losses under load (a renewal that misses its window by a scheduler hiccup drops a healthy room); longer means the reverse. Ten seconds, renewed every three, is the default; the chaos run's budget is expressed in ttl so the test moves with the dial.

5 · The fence: the check and the write are one statement

Section 2 ended with the argument: a process frozen past its lease cannot know it, so the storage must refuse it. F22 spells the fault out twice: `docker pause` the owner past its lease and resume it; and cut the owner off from Redis while Postgres still answers (a partition, scripted with toxiproxy). In both, the old owner wakes believing it owns the room, with

ops queued and appends in flight, and the journal must end with no gap, no duplicate, and no row from it after the new owner read the journal.

FROM packages/db/migrations/0014_fence.sql

```
-- E7.3 (F22): the fence. A sync node that took a room's lease CLAIMS the document here before it reads the
-- journal: fence_token rises to its lease token (never back down), fence_claim becomes the id of that one
-- opening. The journal append inserts only while fence_claim is still the appender's, in the SAME statement
-- that locks this row (FOR UPDATE): an owner frozen past its lease that runs again cannot add a row after a
-- newer owner's claim, and a row it added before the claim is in the journal the newer owner then replays.
-- The token is kept here, not only in Redis: a flushed Redis restarts its counter at 1, so the next acquire
-- is seeded from fence_token, and tokens keep rising across the flush.
alter table documents
  add column fence_token bigint not null default 0 check (fence_token >= 0),
  add column fence_claim uuid;
```

FROM packages/db/src/index.ts

```
/** E7.3: the largest lease token that ever claimed this document (0: none); undefined when there is no such
document. */
fence(orgId: string, documentId: string): Promise<number | undefined>;
/**
 * E7.3: this opening of the room, under lease `token`, becomes the document's only writer: from now on only
 * appends that carry `claim` land. False: a claim with an equal or larger token was made (this lease is
stale).
 */
claim(orgId: string, documentId: string, token: number, claim: string): Promise<boolean>;
/**
 * E6.1a: journals an accepted op, BEFORE anyone hears of it. Undefined: written. An op: this sender's
 * opId was journaled already, and that is what it became (a resend the room had forgotten). Rejects
 * when the op is not durable: the database is away, the seq is taken (a second writer), the document is
gone,
 * or (E7.3) `claim` is no longer the document's: `Fenced`. No claim: only a document never claimed takes
it.
 */
append(orgId: string, documentId: string, op: SequencedOp, claim?: string): Promise<SequencedOp |
undefined>;
```

FROM packages/db/src/index.ts

```
/** E7.3: the append was refused because a newer owner claimed the document. This room must stop writing. */
export class Fenced extends Error {}
```

FROM packages/db/src/index.ts

```
async fence(orgId, documentId) {
  if (!isId(orgId) || !isId(documentId)) return undefined;
  return (await one(z.object({ fence_token: count }), "select fence_token from documents where org_id =
$1 and id = $2", [orgId, documentId]))?.fence_token;
},
async claim(orgId, documentId, token, claim) {
  if (!isId(orgId) || !isId(documentId) || !isId(claim)) return false;
  // '<', not '<=': two openings under one token (a flushed Redis issuing it again) cannot both claim.
  const claimed = await pool.query("update documents set fence_token = $3, fence_claim = $4 where org_id
= $1 and id = $2 and fence_token < $3", [orgId, documentId, token, claim]);
  return claimed.rowCount === 1;
},
async append(orgId, documentId, { seq, opId, actor, op }, claim) {
  if (!isId(orgId) || !isId(documentId)) throw new Error("journal: not a document id");
  try {
    // From the document's row, so that a document of another org (or one deleted meanwhile) takes no
op.

    // The fence is in the same statement (F22): the row must still name this room's claim, and FOR
```

```

UPDATE
    // locks it, so a claim either waits for this insert (and its reader sees the row) or comes first
    (and
        // Postgres re-checks the WHERE against the claimed row: nothing is inserted). Never read-then-
        write.
        const written = await pool.query(
            `insert into op_journal (document_id, org_id, ${JOURNAL_COLUMNS}) select id, org_id, $3, $4, $5,
            $6, $7, $8 from documents where org_id = $1 and id = $2 and fence_claim is not distinct from $9 for update`,
            [orgId, documentId, seq, opId, actor.kind, actor.id, actor.runId ?? null, JSON.stringify(op),
            claim ?? null],
        );
        if (written.rowCount !== 1) {
            // Only telling the two refusals apart, after the fact: nothing was written either way.
            const exists = await pool.query("select 1 from documents where org_id = $1 and id = $2", [orgId,
            documentId]);
            throw exists.rowCount === 1 ? new Fenced("journal: fenced, a newer owner claimed the document") :
            new Error("journal: the document is gone");
        }
        return undefined;
    } catch (err) {
        // Two unique keys, told apart by NAME: 23505 alone cannot say whether this is a resend or a rival
        writer.
        if (err instanceof Error && "constraint" in err && err.constraint === "op_journal_op") {
            const original = await one(JournalRow, `select ${JOURNAL_COLUMNS} from op_journal where
            document_id = $1 and actor_id = $2 and op_id = $3`, [documentId, actor.id, opId]);
            if (original) return original;
        }
        throw err;
    }
},

```

Three statements, and the whole feature is in how they relate. **fence** reads the largest token that ever claimed the document; the acquire in section 1 takes it as the *floor*, so a token issued after a Redis flush is still above every token the journal has seen. **claim** is how an opening becomes the document's only writer: one **update** that moves **fence_token** up to the lease token and sets **fence_claim** to a fresh random id, guarded by **fence_token < \$3**. Strictly less, not less-or-equal: two openings under one token (a node paused between acquire and claim, then a flush that reissues the number) cannot both claim. **append** is one statement: **insert ... select ... from documents where ... fence_claim is not distinct from \$9 for update**. If the row still names this opening's claim, the row is written; if not, zero rows, and the store throws **Fenced**.

Why the check must be *in* the statement rather than before it is the naive control in **fence.int**, and it is exactly the interleaving a pause produces: A reads the claim (still its own), freezes, B claims and reads the journal, A wakes and inserts. A read-then-write in app code lets that row in, one past what B read, so B's room never has it and B's next append collides on the seq. In one statement Postgres does the work. **FOR UPDATE** takes a row lock on the documents row, so an append and a claim on the same document serialise: the claim waits for an in-flight insert to commit (and B's replay then contains that row, correctly, because it was written under a claim that was still valid), or the claim commits first, and the append, which was blocked on the lock, *re-evaluates its WHERE against the updated row* (Postgres's behaviour under **READ COMMITTED** when a locked row changed) and inserts nothing. The test "an append racing an uncommitted claim waits for it, then is refused" is that second branch, with two real connections.

is not distinct from is SQL's null-safe equality: with no lease (one node, the tests) the append carries no claim, **\$9** is **NULL**, and **NULL = NULL** would be unknown, never true; **IS NOT DISTINCT FROM** is true, so an unclaimed append lands only on a document never claimed, which is the right rule for a single node that has never had a rival.

FROM **packages/db/src/fence.int.test.ts**

```

// integration:append-stale-token-rejected (E7.3, F22). Two owners of one document: A (lease token 1) is the
// zombie, B (token 2) took the room after A's lease ran out. B claims the document, reads the journal, and
numbers

```

```
// on from there. Whatever A tries after B's claim must be refused, in a REAL race, and the journal must end
with no
// gap and no duplicate seq. The naive control (read the claim in app code, then insert) is shown to let A's
row in.
```

FROM packages/db/src/fence.int.test.ts

```
test("an append racing an uncommitted claim waits for it, then is refused: the check and the insert are one
statement", async () => {
  const store = t.db.documentStore();
  const doc = await t.createDocument("Fence");
  const a = randomUUID();
  await store.claim(doc.orgId, doc.id, 1, a);
  const b = new Client({ connectionString: TEST_DATABASE_URL, options: { '-c search_path=${t.schema}' } });
  await b.connect();
  try {
    await b.query("begin");
    await b.query("update documents set fence_token = 2, fence_claim = $2 where id = $1", [doc.id,
randomUUID()]);
    // A read its claim BEFORE B's update committed: a read-then-write would insert now. The fenced append
waits.
    let settled = false;
    const late = store.append(doc.orgId, doc.id, row(1, "a"), a).finally(() => { settled = true; });
    late.catch(() => undefined);
    await new Promise((resolve) => setTimeout(resolve, 300));
    expect(settled).toBe(false);
    await b.query("commit");
    await expect(late).rejects.toBeInstanceOf(Fenced);
  } finally {
    await b.end();
  }
  expect(await seqs(doc.id)).toEqual([]);
});
```

FROM packages/db/src/fence.int.test.ts

```
test("naive control: the claim read in app code, then an insert, lets the zombie's row in after the new owner
read the journal", async () => {
  const store = t.db.documentStore();
  const doc = await t.createDocument("Naive");
  const a = randomUUID();
  await store.claim(doc.orgId, doc.id, 1, a);
  // Read-then-write, with the new owner's takeover landing between the read and the write: the one
interleaving
  // a real pause produces (A reads, is frozen past its lease, B claims and reads, A resumes and writes).
  const current = ((await t.rawQuery("select fence_claim from documents where id = $1", [doc.id])) as { rows:
[ { fence_claim: string } ] }).rows[0].fence_claim;
  expect(current).toBe(a); // A believes it may write
  expect(await store.claim(doc.orgId, doc.id, 2, randomUUID())).toBe(true);
  const read = (await store.since(doc.orgId, doc.id, 0)).at(-1)?.seq ?? 0;
  const late = row(read + 1, "a");
  await t.rawQuery("insert into op_journal (document_id, org_id, seq, op_id, actor_kind, actor_id, op) values
($1, $2, $3, $4, 'user', 'a', $5)", [doc.id, doc.orgId, late.seq, late.opId, JSON.stringify(late.op)]);
  // The property the fenced append keeps is broken: a zombie row past what the new owner read. B's own next
op
  // now collides on that seq, or (had B numbered first) A's op is lost from B's room and every peer's view.
  expect((await seqs(doc.id)).filter((r) => r.actor === "a" && r.seq > read)).toHaveLength(1);
  // The same interleaving through the fenced append: refused.
  await expect(store.append(doc.orgId, doc.id, row(read + 1, "a"), a)).rejects.toBeInstanceOf(Fenced);
});
```

Why a claim id and not "token at least the high-water mark". The textbook fence compares tokens: the storage remembers the largest it has seen and refuses anything smaller. Two things about this repo made that insufficient. The

counter lives in Redis, and a flushed Redis restarts it at 1, so the floor was needed to keep tokens rising at all. And even with the floor, one token can be issued twice (the pause-then-flush case), so the claim is a random id of *this one opening*, and an append compares the exact id: the loser of two openings under an equal token cannot write either. The token still matters, as the thing that orders claims and as the floor; the claim id is what an append carries.

FROM `packages/lease/src/index.int.test.ts`

```
test("a floor (the journal's fence, E7.3) keeps tokens rising after the counter is lost, as a flushed Redis loses it", async () => {
  const documentId = randomUUID();
  const first = await a.acquire(documentId, "sync", 41);
  expect(first.holder.token).toBe(42);
  await a.release(documentId, first.holder);
  await raw.del(`${prefix}lease-token:${documentId}`); // FLUSHALL, for this one document
  const next = await b.acquire(documentId, "sync-2", 42);
  expect(next.holder.token).toBe(43);
  await b.release(documentId, next.holder);
  expect((await a.acquire(documentId, "sync", 5)).holder.token).toBe(44); // a floor below the counter changes nothing
});
```

The node's side. The claim happens in `open()`, after the lease and before the journal is read: from that moment only this opening's appends land, and every row a previous owner wrote before it is in what `load()` replays. A lost claim at open means a newer lease exists: 4409. A database that cannot answer the fence or the claim: 4503, try again.

FROM `apps/sync/src/server.ts`

```
// The fence (F22), BEFORE the journal is read: from here on only this opening's appends land, and every
row a
// previous owner added before this moment is in what load() replays. A lost claim: a newer lease exists.
const claim = randomUUID();
let claimed = true;
if (store) {
  try {
    claimed = await store.claim(orgId, documentId, holder.token, claim);
  } catch {
    await release();
    return { closeCode: CLOSE.unavailable };
  }
}
const loaded = claimed ? await load(documentId, orgId, { claim, onFenced: () => { drop("fenced by a newer owner's claim"); } }) : { closeCode: CLOSE.roomElsewhere };
if ("closeCode" in loaded || !keeper.held) {
  await release();
  if (!("closeCode" in loaded)) loaded.snapshot?.stop();
  return "closeCode" in loaded ? loaded : { closeCode: CLOSE.roomElsewhere };
}
opened.room = owned(loaded, release);
return opened.room;
}
```

FROM `apps/sync/src/fence.ts`

```
import { Fenced } from "@noon/db";

/**
 * A room's journal append that also tells the server when it was FENCED (E7.3, F22): a newer owner claimed the
 * document, so this room must stop at once and send its peers to that owner. The refusal itself still reaches
 * the room (which answers "unavailable" and applies nothing); any other failure is only an outage, left to
 * E6.1b's read-only mode. `onFenced` runs once, however many queued appends are refused after it.
 */
```

```
export function watchFence<A extends unknown[], R>(append: (...args: A) => Promise<R>, onFenced: () => void):
(...args: A) => Promise<R> {
  let fenced = false;
  return async (...args) => {
    try {
      return await append(...args);
    } catch (err) {
      if (err instanceof Fenced && !fenced) {
        fenced = true;
        onFenced();
      }
      throw err;
    }
  };
}
```

FROM `apps/sync/src/server.ts`

```
// A timed-out append may still land later: the room treats it as failed, and recover() replays it if it
did.
// A fenced one never lands: the room is dropped, not healed (a newer owner is writing).
let fenced = false;
// An edit reaches the other browsers only after these answer: a slow one is logged, with which it was.
const timed = reportSlow(SLOW_JOURNAL_MS, (what, ms) => { log(documentId, `journal ${what} took
${String(ms)} ms`, "warn"); });
const journal = {
  append: watchFence((op: SequencedOp) => timed("append", bounded(store.append(orgId, documentId, op,
fence?.claim))), () => {
    fenced = true;
    fence?.onFenced();
  }),
  find: (actorId: string, opId: string) => timed("find", bounded(store.find(orgId, documentId, actorId,
opId))),
  everAdded: (nodeId: string) => timed("everAdded", bounded(store.everAdded(orgId, documentId, nodeId))),
  since: (after: number) => timed("since", bounded(store.since(orgId, documentId, after))),
};
```

`watchFence` is a higher-order function typed with a tuple rest parameter, the same shape as lesson 6's `guard`: it wraps the `append`, keeps its signature, and tells the server *once* when a `Fenced` comes through, however many queued ops are refused behind it. The distinction it draws is the design decision of the bead: **a fence is not an outage**. Lesson 6's read-only mode is for a journal that will answer again; a fenced room will never be allowed to write again, so it must not sit in a heal loop pinging the database for ever. `onFenced` calls the same `drop` a lost lease calls: 4409 to every socket, snapshots stopped, room forgotten. `onReadOnly` checks `fenced` first so the heal loop is never started. The refusal itself still reaches the room, which answers "unavailable" to a socket that is already closing and applies nothing.

FROM `apps/sync/src/fence.test.ts`

```
test("a fenced append is still refused to the room, and says so to the server exactly once", async () => {
  const onFenced = vi.fn();
  const append = watchFence((seq: number) => (seq > 1 ? Promise.reject(new Fenced("fenced")) :
Promise.resolve(undefined)), onFenced);
  await expect(append(1)).resolves.toBeUndefined();
  expect(onFenced).not.toHaveBeenCalled();
  await expect(append(2)).rejects.toBeInstanceOf(Fenced);
  await expect(append(3)).rejects.toBeInstanceOf(Fenced); // a queued op refused after it: not a second drop
  expect(onFenced).toHaveBeenCalledTimes(1);
});

test("an outage is not a fence: the room goes read-only and the server keeps the room", async () => {
  const onFenced = vi.fn();
  const append = watchFence(() => Promise.reject(new Error("journal timed out")), onFenced);
  await expect(append()).rejects.toThrow("timed out");
});
```

```
expect(onFenced).not.toHaveBeenCalled();
});
```

FROM apps/sync/src/server.ts

```
/** The room's last snapshot, once the ops still in its queue are handled. False: not stored (logged). */
async function snapshotNow(opened: Owned): Promise<boolean> {
  // A room that lost its lease writes nothing more: the new owner is the only writer (the journal is
  fenced).
  if (opened.lost) return false;
  await opened.room.settled();
  return (await opened.snapshot?.take()) ?? true;
}
```

Two more places had to learn about `lost`. A room that lost its lease or was fenced writes *no last snapshot*: the new owner is the only writer, and a snapshot from the old one could name a document the journal has moved past. And the shutdown path releases every lease after its snapshot, so that a clean stop moves rooms at once rather than a ttl later.

The runs. Both faults are scripted against the compose stack. `docker pause` freezes every thread of the container; when it is unpaused, the process continues from the instruction it was on, with its sockets still open and its in-flight promises still pending, which is the most faithful zombie there is. The partition uses toxiproxy in the path between each node and Redis, one listener per node, so *one* node can be cut off from Redis with a `timeout` toxic (bytes stop, connections stay open) while it still reaches Postgres, which is the case where the fence is the only thing standing.

FROM docker-compose.yml

```
# E7.3: each sync node reaches Redis through this proxy, one listener per node
(scripts/chaos/toxiproxy.json), so
# that chaos:partition-fenced can cut ONE node off Redis (bytes stop, connections stay open) while it still
# reaches Postgres. A plain pass-through otherwise. The control API is published on 127.0.0.1 for the chaos
runs.
# ponytail: in the path of every sync->Redis call, in every run; ceiling: one more hop and one more
container.
# Upgrade: a chaos-only compose override that inserts it.
redis-proxy:
  image: ghcr.io/shopify/toxiproxy:2.12.0
  command: ["-host=0.0.0.0", "-config=/toxiproxy.json"]
  volumes: ["/scripts/chaos/toxiproxy.json:/toxiproxy.json:ro"]
  ports: ["127.0.0.1:${TOXIPROXY_PORT:-8474}:8474"]
  depends_on:
    redis: { condition: service_healthy }
  healthcheck:
    test: ["CMD", "/toxiproxy-cli", "list"]
    interval: 2s
    timeout: 2s
    retries: 30
```

FROM scripts/chaos/toxiproxy.json

```
[
  { "name": "sync", "listen": "0.0.0.0:6379", "upstream": "redis:6379", "enabled": true },
  { "name": "sync-2", "listen": "0.0.0.0:6380", "upstream": "redis:6379", "enabled": true }
]
```

FROM scripts/chaos/fenced.ts

```
// chaos:zombie-fenced and chaos:partition-fenced (E7.3, SPEC F22). Against the REAL compose stack (two sync
nodes,
// each reaching Redis through the `redis-proxy` toxiproxy):
// ./init.sh (or: docker compose up -d), then node scripts/chaos/fenced.ts zombie|partition (ROUNDS=2
```

```

by default)
// Each round, on a fresh document: a person and the AI (both routed by /session, as in E7.2) get a few edits
// acknowledged, and a WITNESS, a bare WebSocket that never reconnects, joins the owner's room. Then, under a
burst:
//   zombie:   the owner is `docker pause`d past its lease (ttl + 2 s after another node took the room), then
resumed;
//   partition: the owner is cut off from Redis ONLY (toxiproxy's timeout toxic: bytes stop, connections stay
open,
//               Postgres still answers), for as long, then the cut is healed.
// Either way: the peers must find the other node through /session, the lease must move to it with a larger
token,
// the old owner must close its sockets (the witness is dropped, and the old owner logs why) and never append
again (every row
// of the journal is one the peers made, once; seqs 1..n, no gap, no duplicate: scripts/chaos/no-loss.ts).
Edits made
// AFTER the old owner is back must land too. How many appends the fence itself refused is read from the old
owner's
// log and reported (a paused owner usually notices its lost lease first; the fence is for the ones that do
not).
// Pitfalls (SPEC §4a): the fault opens while the workload is on the wire; waits poll the observable; it
repeats; the
// fault is undone in `finally`. Prints one JSON line; exit 0 = PASS. The org is deleted at the end.

```

FROM scripts/chaos/fenced.ts

```

// The toxiproxy proxy of a node is named as the node (scripts/chaos/toxiproxy.json). Both directions stop.
const STREAMS = ["upstream", "downstream"] as const;
async function toxics(node: string, cut: boolean): Promise<void> {
  for (const stream of STREAMS) {
    const res = cut
      ? await fetch(`${toxiproxy}/proxies/${node}/toxics`, { method: "POST", body: JSON.stringify({ name:
`cut-${stream}`, type: "timeout", stream, toxicity: 1, attributes: { timeout: 0 } }) })
      : await fetch(`${toxiproxy}/proxies/${node}/toxics/cut-${stream}`, { method: "DELETE" });
    if (!res.ok && !(res.status === 404 && !cut)) throw new Error(`toxiproxy ${cut ? "cut" : "heal"} ${node}
${stream} -> ${String(res.status)}`);
  }
}
async function open(node: string): Promise<void> {
  if (fault === "zombie") compose("pause", node);
  else await toxics(node, true);
}
async function undo(node: string): Promise<void> {
  if (fault === "zombie") compose("unpause", node);
  else await toxics(node, false);
}

```

FROM scripts/chaos/fenced.ts

```

/**
 * A peer of the owner's room that never reconnects (its session answers once, then "give up") and never waits
for
 * anything (so its own silence watchdog never fires): it stops being live only when the old owner closes it.
 */
async function witness(session: () => Promise<{ wsUrl: string; token: string }>): Promise<{ closed: () =>
boolean; close: () => void }> {
  let asked = false;
  const peer = connectPeer({ manifest, session: async () => { if (asked) return null; asked = true; return
session(); } });
  await until(() => peer.status === "live", "the witness joins the owner's room", 15_000);
  return { closed: () => peer.status !== "live", close: () => { peer.close(); } };
}

```

FROM `scripts/chaos/fenced.ts`

```
edit(20); // on the wire...
await open(victim); // ...and the owner freezes, or loses Redis, under it
ledger.fault();
// The burst above is usually ALL acknowledged before the fault bites (`docker pause` blocks this loop
while the
// owner answers), and a peer times only what it waits for: with nothing pending it would never notice a
frozen
// owner. These are made AFTER the fault, so both peers wait on it and their silence watchdog fires.
edit(3);
await until(() => all.every([], p] => p.status !== "live"), "both peers see the owner gone", 30_000);
edit(5); // made while the room is moving: held, sent after the next welcome

let after: ReturnType<typeof holderOf>;
await until(() => { after = holderOf(doc); return after !== undefined && after.node !== victim; },
"another node takes the room", ttlMs * 2 + 30_000);
if (!after || after.token <= before.token) throw new Error(`the new owner's token ${String(after?.token)}
is not above the old one's ${String(before.token)}`);
await until(() => all.every([], p] => p.status === "live" && p.pendingCount === 0), "both peers
reconnected through /session, nothing pending", 30_000);
// Past its lease for certain, then back: a zombie that still believes it owns the room.
await new Promise((resolve) => setTimeout(resolve, 2000));
await undo(victim);
const back = victim;
victim = undefined;
await until(() => watcher?.closed() === true, `the old owner ${back} closes its sockets`, ttlMs + 10_000);

edit(10); // the old owner is running again: none of this may reach it
await ledger.settle(15_000);
await until(() => all.every([], p] => p.seq === peers.person.seq && p.pendingCount === 0), "both peers at
the same seq", 10_000);
```

The *witness* is the observable for "it closes its sockets": a bare peer that never reconnects and never times out on its own, so the only way it stops being live is the old owner closing it. The run also reads the old owner's log for two lines: that it gave the room up (`lease N lost` or `fenced`), and how many appends the fence itself refused. A paused owner usually notices its lost lease first, from its clock, and the count is often zero; the fence is for the runs where it does not, and the report says which happened. Edits made *after* the old owner is back must also land, on the new owner, which is the last thing the ledger checks.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>insert ... select ... where ... for update</code> : the check, the lock and the write in one statement	the same SQL through JDBC; JPA's <code>@Version</code> is the row-level cousin, not this	the same SQL with <code>psycopg</code>
<code>fence_claim</code> is not distinct from <code>\$9:null-safe equality</code>	the same SQL; <code><=></code> in MySQL	the same SQL
<code>class Fenced extends Error {}</code> , told apart with <code>instanceof</code>	a checked exception type	a custom exception class
<code>watchFence<A extends unknown[], R>(append, onFenced)</code> : a wrapper that keeps the signature	a decorator with one wrapper per arity	a decorator with <code>ParamSpec</code>
<code>randomUUID()</code> as an opening's identity	<code>UUID.randomUUID()</code>	<code>uuid.uuid4()</code>
<code>new Client({ options: "-c search_path=..." })</code> : a second real connection in a test	a second <code>DataSource</code>	a second <code>psycopg.connect</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The fence in app code, before the insert, was rejected by writing its failure down as a test.

The first sketch read `fence_claim`, compared it, and inserted. It is the shape every ORM invites, and the naive control in `fence.int` exists to keep it out: it performs the read, then the takeover, then the insert, and asserts that the zombie's row *is* in the journal. A control that shows the wrong design failing is worth more than a comment saying it would, because the next person who wants to "simplify" the statement has a red test to explain. Drill 2 plants this shape.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Seeding the Redis counter only at sync start was rejected.

Reading `fence_token` once when the node boots and `SET` ting the counter would handle a flush that happened while the node was down. A flush while it runs would still reissue tokens. Taking the floor on *every* acquire, inside the same script, costs one `GET` per acquisition and closes the window entirely; the integration test deletes the counter mid-test and asserts 43 follows 42.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Two openings under one token, and why "less than" is not "less than or equal".

A node acquires token 7, is paused before it claims, Redis is flushed, another node acquires and gets 7 again from a floor of 6, claims, and reads the journal. The paused node wakes and claims with 7. Under `<=` it would succeed, and two rooms would each hold a valid claim; under `<` it is refused and answers 4409. The exact-claim compare in the append is the second half: even if both somehow held token 7, only one holds the claim id the row names.

The distributed-systems shape, in one sentence: the lease says who *should* write, the clock says how long they may *believe* it, and the storage says whose writes actually *land*, by comparing a token the writer received when it became the owner. Chubby's sequencers, ZooKeeper's `zxid`, Kafka's producer epoch and etcd's revision are the same third piece with other names. A design with only the first two is the one Kleppmann's essay takes apart.

6 · Trade-offs and comparisons: where this design sits

An interviewer will ask why Redis and not ZooKeeper or etcd, and what breaks when. Here is the honest map.

QUESTION	THIS REPO	ZOOKEEPER / ETCD	NOTES
Where the lease lives	Redis: one node, no consensus, fast, loses everything on a flush or crash	ZooKeeper (ZAB) or etcd (Raft): a majority must agree; survives a minority failing	the same two choices; Consul sessions are a third
What losing it costs	leases are taken again; the token floor in Postgres keeps tokens rising; queues are rebuilt from the <code>jobs</code> table (SPEC §4)	a lost quorum stops all acquisitions until it returns	
The fencing token	<code>INCR</code> per document, floored by Postgres	etcd's <code>revision</code> ; ZooKeeper's <code>zxid</code> or the sequential node's number	
Who checks the token	Postgres, in the append statement	the storage the lock protects, always; ZooKeeper and etcd only <i>issue</i> it	
Liveness	a heartbeat key with a short PX; a hint only	an ephemeral node that vanishes with the session; a lease that lapses without keep-alive	
Java's Redisson <code>RLock</code>	same Lua shape; a watchdog renews every <code>ttl/3</code> ; no fencing token , so a paused holder's writes are not refused by anything		
RedLock (several Redis masters)	not used: Kleppmann's critique applies (it still relies on timing), and one Redis with a fence in the storage is both simpler and safer for this workload		

Why Redis is enough here. The lease is a *performance* mechanism in this design, not a *safety* one. Its job is to make sure that, almost always, exactly one node believes it owns a room, so that peers are not bounced and ops are not refused. Safety, the guarantee that no two writers ever both land rows for one document, is Postgres's job through the fence, and Postgres was already the truth for the journal. A consensus system would make the lease itself more reliable, at the cost of a second quorum to run, and it would still need the fence in the storage, because a leader elected by Raft can also be paused. Kleppmann's point is that the lock service is the wrong place to look for safety whatever it is built on.

What is not solved. A room moves only when its owner dies or lets go; there is no rebalancing, so a node that happened to be picked for many free rooms keeps them. The pick for a free room is random among live nodes, with no load signal. Both are named ponytails with an obvious upgrade (a room count per node in Redis, read by `pick`). Failover takes one `ttl`, which is visible: peers show "offline" for up to ten seconds. And `redis-proxy` is in the path of every run, not only chaos runs, one hop and one container that a compose override would move aside.

Redis flushed while everything runs. Every lease disappears at once. Every owner's next renewal is refused (the key is gone), so every room drops its peers within a third of a `ttl`; every peer asks `/session`, which sees no owner and picks a live node (or, with no heartbeats written yet, any node); the node acquires with a floor read from `fence_token`, so its token is above every claim the journal has seen, claims, replays, and opens. No op is lost: acknowledged ops are in the journal, in-flight ones are held by peer-client and re-offered after the next welcome. SPEC §4's row, *Redis lost: leases re-acquired; no document data lost*, is this paragraph.

The interview answer in one breath: one lease per document in Redis with a monotonic token from the same script; holders trust it for 0.9 ttl from the send on their own monotonic clock; heartbeats route but never decide; a dead lease is waited out, never stolen; and the write path in Postgres checks a per-opening claim in the same statement that inserts, so a zombie's writes are refused whatever it believes. Consistency over availability for up to one ttl, by choice, because two rooms for one document cannot be repaired.

7 · Testing two nodes without Docker, and then with it

The split from lesson 6 holds: every decision has a pure test that ran on every commit, and every I/O rule has an integration or chaos test that is written and unrun. Four shapes are new.

A fake clock that the sleep advances. `takeLease` 's tests inject `now`, `sleep` and `acquire`; the `acquire` costs one millisecond of fake time and answers "free" once the clock passes `expiresAt`; the `sleep` records what it was asked and moves the clock. So "waited out, taken within a poll of expiry" is an assertion on exact numbers, and the whole suite runs in milliseconds.

Two real servers in one process. `useTwoNodes` starts two `startSyncServer` s on free ports with two `createLeases` clients under a random key prefix, and hands the test a `route` that is the api's router with an injectable first pick. `roomAt` asks both servers who has peers for a document and *throws* if both do, which is the F20 invariant as a helper.

FROM `apps/sync/src/testing.ts`

```
/**
 * Two sync nodes sharing one Redis (keys under a prefix of this file's own), for the F20 tests: each node
 * holds
 * rooms only under its lease. 'route' is what the api's /session answers: the owner's address, else 'pick'.
 * Redis is the compose stack's (./init.sh).
 */
export function useTwoNodes({ ttlMs = 600 }: { ttlMs?: number } = {}) {
  const prefix = `test-nodes-${randomUUID()}`;
  const ids = ["node-a", "node-b"] as const;
  let state: { servers: [RunningSyncServer, RunningSyncServer]; leases: Leases[]; route: (documentId: string,
  pick?: (typeof ids)[number]) => Promise<string> } | undefined;
  beforeAll(async () => {
    const leases = ids.map(() => createLeases({ redisUrl: TEST_REDIS_URL, ttlMs, prefix }));
    await Promise.all(leases.map((each) => each.ready()));
    const servers = await Promise.all(ids.map((nodeId, i) => startSyncServer({ port: 0, secrets:
    [TEST_SECRET], lease: { leases: leases[i] as Leases, nodeId } }))) as [RunningSyncServer, RunningSyncServer];
    const nodes = { kind: "many", nodes: new Map(ids.map((id, i) => [id, (servers[i] as
    RunningSyncServer).url])) } as const;
    const lookup = leases[0] as Leases;
    state = {
      servers, leases,
      route: (documentId, pick) => syncRouter({ nodes, owner: (id) => lookup.owner(id), alive: (nodeIds) =>
      lookup.alive(nodeIds), ...(pick === undefined ? {} : { pick: () => pick }) })(documentId),
    };
  });
}
```

A second database connection. The race test in `fence.int` needs an *uncommitted* claim to be visible to nothing and blocking to one thing, which a pool cannot express: it opens a raw `pg.Client` on the test schema, begins a transaction, updates the row, and only then commits, while the store's append sits on the lock. `settled` being false after 300 ms is the assertion that the append waited rather than returned.

FROM `packages/db/src/fence.int.test.ts`

```
test("real race, 20 rounds: after the claim no zombie append lands, and the journal has no gap and no
duplicate seq", async () => {
  const store = t.db.documentStore();
```

```

for (let round = 0; round < 20; round++) {
  const doc = await t.createDocument(`Race ${String(round)}`);
  const a = randomUUID();
  await store.claim(doc.orgId, doc.id, 1, a);
  const [refusals, read] = await Promise.all([zombie(store, doc, a), takeOver(store, doc, round % 5)]);
  const journal = await seqs(doc.id);
  expect(journal.map((r) => r.seq).toEqual(journal.map((_, i) => i + 1))); // 1..n: no gap, no duplicate
  expect(journal.filter((r) => r.actor === "a").every((r) => r.seq <= read)).toBe(true); // A only before B
  read
  expect(journal.filter((r) => r.actor === "b")).toHaveLength(20);
  expect(refusals.at(-1)).toBeInstanceOf(Fenced); // the zombie was stopped by the fence, not by running out
}
});

```

A witness. The chaos runs need to observe "the old owner closed its sockets", and a normal peer hides that by reconnecting. The witness is `connectPeer` with a `session` that answers once and then `null` ("give up"), so its status leaving `live` can only mean the server closed it.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>sleep: (ms) => { slept.push(ms); clock += ms; return Promise.resolve(); }</code>	a fake <code>Sleeper</code> that advances a fake <code>Clock</code>	a fake sleep coroutine
<code>as [RunningSyncServer, RunningSyncServer] : a tuple from Promise.all of a mapped array</code>	an array of two, indexed	a tuple from <code>asyncio.gather</code>
<code>new Client({ options: "-c search_path=..." })</code>	a JDBC URL with <code>currentSchema</code>	<code>psycopg.connect(options="-c search_path=...")</code>
<code>if (asked) return null; asked = true; : a session that answers once</code>	a one-shot <code>Supplier</code>	a closure over a flag
<code>execFileSync("docker", ["compose", "pause", node])</code>	<code>ProcessBuilder</code>	<code>subprocess.run</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The integration test's kill had to be unclean to test anything.

A first draft of the failover test called `server.close()`, which snapshots and *releases* every lease; the room moved instantly and the test was green for the wrong reason. Closing the node's Redis client first, so that the release fails and the heartbeat stops, is what makes the test wait out a real lease. "Green for the wrong reason" is the failure mode of every test of a failure: check what the fault actually removed.

Honest status, repeated here because it matters: the unit tests (holder parsing, `keepLease`, `parseSyncNodes`, `syncRouter`, `takeLease`, `watchFence`, peer-client's reroute, the routing config fields) ran green on every commit and were mutation-checked. The integration suites (`lease index.int`, `api session.int`, `one-room-globally.int`, `one-room-join-race.int`, `failover.int`, `packages/db fence.int`) and the three chaos runs (`kill-owner-failover`, `fenced zombie`, `fenced partition`) are written, reviewed and committed, and have not been run. The next section is what running them looks like.

8 • Two nodes by hand: what to run once the stack is up

The exact commands and what each should show. The dev stack now runs `sync` and `sync-2` (host ports 3001 and 3003), `redis-proxy` (toxiproxy, control API on 127.0.0.1:8474), and the api's `SYNC_PUBLIC_URL` is the two-node table. Where a document id is needed it is the `?doc=` in the canvas's address bar.

0 • BRING THE STACK UP

```
./init.sh                                # two sync nodes; migrate runs 0014 (fence_token, fence_claim)
docker compose ps                         # sync, sync-2, redis-proxy healthy
open http://localhost:5173/?user=you@example.com    # New document; Add Stack; note ?doc=<id>
```

1 • WHO OWNS THE ROOM (F20)

```
docker compose exec -T redis redis-cli --no-auth-warning GET lease:<id>      # "1:sync" or "1:sync-2": token
1, the owner
docker compose exec -T redis redis-cli --no-auth-warning MGET node:sync node:sync-2  # both "1": both beating
curl -s -X POST -H 'x-dev-user: you@example.com' localhost:3000/documents/<id>/session | jq .wsUrl  # the
owner's address
# Open the same document in a second tab: it lands in the same room (edits appear in both). Open a NEW
document: it may land on the other node.
docker compose exec -T postgres psql -U noon -d noon -c "select fence_token, fence_claim from documents where
id = '<id>'"    # 1, and this opening's claim
```

2 • KILL THE OWNER (F21)

```
docker compose kill --signal SIGKILL sync    # (or sync-2: whichever GET lease named). Keep editing:
"offline", edits queue locally
docker compose exec -T redis redis-cli --no-auth-warning MGET node:sync node:sync-2  # the dead one is nil
within ttl/3 (about 3 s)
# Within about 10 s (one LEASE_TTL_MS), without a reload: "live", then "saved"; the lease now names the other
node with token 2:
docker compose exec -T redis redis-cli --no-auth-warning GET lease:<id>      # "2:sync-2"
docker compose start sync              # back for the next run; it owns nothing until a free room's
first peer lands on it
```

3 • THE ZOMBIE (F22)

```
docker compose pause <owner>              # the owner freezes. Edit in the tab: "offline"; the other node
takes the room after one ttl
docker compose unpause <owner>              # a moment later its log says the room was given up:
docker compose logs --no-color <owner> | grep -E 'lease [0-9]+ (lost|fenced)'\
docker compose exec -T postgres psql -U noon -d noon -c \
    "select count(*), count(distinct seq), max(seq) from op_journal where document_id = '<id>'"    # all equal:
no gap, no duplicate
```

4 • THE PARTITION (F22, THE OTHER FAULT)

```
curl -s -X POST localhost:8474/proxies/sync/toxics -d
'{"name":"cut","type":"timeout","stream":"downstream","toxicity":1,"attributes":{"timeout":0}}'
# node "sync" can no longer hear Redis; Postgres still answers
it. Its renewals fail; after 0.9 ttl it drops its rooms
curl -s -X DELETE localhost:8474/proxies/sync/toxics/cut
LEASE_TTL_MS=3000 docker compose up -d sync sync-2    # to watch faster: failover in about three seconds
```

5 • THEN THE CHECKS THAT PROVE IT WITHOUT HANDS

```
pnpm exec vitest run --config vitest.integration.config.ts packages/lease/src/index.int.test.ts
apps/api/src/session.int.test.ts apps/sync/src/one-room-globally.int.test.ts apps/sync/src/one-room-join-
race.int.test.ts apps/sync/src/failover.int.test.ts packages/db/src/fence.int.test.ts
```

```

node scripts/chaos/kill-owner-failover.ts    # {"chaos":"kill-owner-failover","verdict":"PASS","rounds":
[{"from":"sync","to":"sync-2","movedAfterMs":...,"violations":[]},...]}
node scripts/chaos/fenced.ts zombie          # {"chaos":"zombie-fenced","verdict":"PASS","rounds":
[{"fencedAppends":0,...}]}
node scripts/chaos/fenced.ts partition       # {"chaos":"partition-fenced",...}
make chaos                                  # all five runs, in order; breaks and restores the real stack
make check                                  # the whole gate, as the pre-commit hook runs it

```

FROM Makefile

```

# Outside `make check`: the chaos checks (SPEC §4a) break the REAL compose stack (./init.sh first) and prove
# the system's guarantees hold. Each prints one JSON line and exits non-zero on FAIL.
.PHONY: chaos
chaos: ; node scripts/chaos/postgres-down-read-only.ts && node scripts/chaos/kill-sync-no-loss.ts && node
scripts/chaos/kill-owner-failover.ts && node scripts/chaos/fenced.ts zombie && node scripts/chaos/fenced.ts
partition && node scripts/chaos/kill-worker-resumes.ts && node scripts/chaos/redis-wipe-rebuild.ts

```

What to do when a step does not match this page: the page was cut from the code at commit `f21d6d2`, and the code is right until a test says otherwise. A step that fails is a finding: write it as *sequence* → *false belief* → *consequence* → *smallest fix* (SPEC §4a's form) and file it against the bead that owns the code.

9 • What the type system could not see, epic 7 edition

Everything below compiled under `strict`, passed the linter and the first tests, or was rejected at design time by asking what the types could not ask.

WHAT WENT WRONG, OR WOULD HAVE	WHY	FIX
a token issued to every loser of an acquire race	two round trips look like one operation	<code>INCR</code> only on the branch that does the <code>SET</code> , in one script
a stale holder's <code>DEL</code> deleted the next owner's lease (measured)	a key is a key	compare-and-delete in Lua
a deadline counted from the reply, not the send	both are "now"	<code>sentAt</code> read before the await
a hung renewal kept a room alive	a promise in flight is not an error	the deadline is checked on every read of <code>held</code>
a failed renewal shortened a lease an earlier one had earned	an assignment is an assignment	only <code>true</code> moves <code>validUntil</code>
a wall clock that jumps backwards	<code>Date.now()</code> and <code>performance.now()</code> are both numbers	monotonic only
a frozen process believes it holds the lease (measured, 1.7 s)	no type describes a pause	the fence in the storage
the api took leases for nodes that could not renew them	acquire and renew look independent	the holder is the acquirer
a hash pick sent peers to a dead node for ever	a pure function looks fair	heartbeats as a routing hint
stealing on a missed heartbeat let a paused owner overlap the new one	"not beating" reads as "dead"	wait the lease out; never steal
a second ttl lost to the client's backoff	a retry curve has no budget	the server holds the peer while it waits
a restarted node bounced its own peers (E7.1's ponytail)	its own id looked like a rival	our own dead lease is waited out too
<code>"/sync"</code> swallowed <code>"/sync-2"</code>	a prefix is a prefix	anchored regex keys
an owner not in the routing table looped on 4409	a <code>Map.get</code> returns undefined quietly	throw, loudly
the fence checked in app code let a zombie row in (the naive control)	two statements look sequential	one statement, <code>for update</code>
a flushed Redis reissued token 1	a counter is assumed monotonic	a floor from <code>fence_token</code> on every acquire
two openings under one token both claimed	<code><=</code> reads as harmless	<code><</code> , and an exact claim id
<code>NULL = NULL</code> refused every single-node append	SQL's three-valued logic	<code>is not distinct from</code>
a fenced room healed for ever	a refused append looked like an outage	<code>Fenced</code> drops; <code>onReadOnly</code> checks <code>fenced</code>

a lost room wrote a last snapshot over the new owner's	the snapshot path did not know about leases	<code>if (opened.lost) return false</code>
a clean close killed the failover test's reason to exist	release and death look alike to a test	close the Redis client first
the worker's peers dialled one node directly	a URL compiled fine	<code>syncSessions</code> routes like <code>/session</code>

The pattern, extended once more: lesson 6 said types cannot see whether a write happened or which order two effects took. Epic 7 adds the two that multi-node turns on: types cannot see *time* (a deadline from the wrong instant compiles; a pause is invisible) and cannot see *which process is speaking* (a stale owner's write is the same TypeScript as a live one's). Both were answered the same way as before: put the truth in one place (Redis for who should own, Postgres for whose writes land), read it before you believe anything, compare tokens rather than clocks wherever safety is at stake, and write a test that provokes the exact interleaving.

Your turn: three drills

Do these alone. They live in `drills/lesson-7/` and both coded drills start RED; `sh drills/lesson-7/check.sh` shows your progress. Neither needs Docker.

D1 • Extend a type, end to end. With two nodes, nothing on the screen says which one a canvas landed on. Carry the node id from the lease to the status line: `SessionResponse` gains an optional, validated `nodeId` (`packages/contracts`); `syncRouter` answers `{ wsUrl, nodeId }`, and a single node `{ wsUrl }` with no id at all (`packages/lease/src/lease.ts`); `/session` puts it in the response; peer-client exposes `nodeId` from its *last* session, so a peer sent elsewhere by a 4409 shows where it landed; `onNode(nodeId)` in `apps/web/src/reasons.ts` says "on sync-2"; the canvas shows it after "saved", as text. make check must stay green: the type checker will name every caller of the router's result (the api, the worker's `roomUrl`, `apps/sync/src/testing.ts`, the one-room integration tests, `failover.int`). That list is the drill.

Predict first: why must the field be optional rather than required, and what would a required field force on every deployment with one sync node?

D2 • Fix a planted bug. `drills/lesson-7/d2-fenced-room/room.ts` is the room cut down to the fence (queue, claim, fenced append, drop) with one bug from this chapter planted in it. Its first test freezes the room between two round trips while another node claims the document and reads the journal. Read the failing assertion and say, before you open the code, which row is in the journal that the new owner's room has never seen, and what the new owner's next append does to a journal that holds it. The fix replaces two calls with one.

D3 • Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. A holder could trust its lease until "the renewal's reply arrived plus ttl", read from `Date.now()`. Instead it trusts until "the renewal was sent plus 0.9 ttl" on `performance.now()`, and the doc comment says this is "not safety". Defend all three choices, then say what the fourth mechanism is that makes it safe.

Redis starts its expiry countdown when the command *arrives*, which is after we sent it; so a deadline counted from the send can never outlast Redis's, while one counted from the reply trusts the lease for a whole round trip longer than Redis will keep it, and under a slow network that is a window in which two nodes both

believe they own the room. The 0.9 is the allowance for the two clocks running at slightly different rates.

`performance.now()` is monotonic: it cannot jump when NTP corrects the wall clock, and a backwards jump of `Date.now()` would extend a deadline by exactly the jump. "Not safety" because none of this runs while the process is frozen: a pause of two seconds inside an `await` resumes with the flag still true and the next instruction already chosen, and the learning test measured the other node owning the room 1.7 s before the frozen one could notice. The fourth mechanism is the fence: the storage compares a token the writer received when it became the owner, in the statement that writes, so what a frozen process believes is irrelevant to what lands. The clock bounds *failover time*; the token gives *safety*; neither does the other's job.

▼ 2. When an owner's heartbeat stops, the other node could take the room at once. Instead it sends new peers to a live node that polls until the dead lease expires, up to about one ttl, and never steals. Why, and why is the same rule applied to a node's *own* previous lease?

Because "stopped beating" is not "stopped writing". A killed node does neither; a paused node does neither and then wakes; a node cut off from Redis stops beating and keeps its Postgres connection. Stealing on a missed heartbeat would make the paused case the normal one: two nodes owning a room from the moment of the steal until the old one's clock runs out, and every op the old one accepts in that window an op its peers were told about that the fence must then refuse. Waiting for the expiry keeps the invariant that at most one node believes it owns the room at any moment, pauses aside, and turns the fence into the rare backstop it should be. Liveness therefore only chooses where a peer is *sent* (a wrong guess costs one 4409), and the lease alone says who owns. A node's own dead lease is the same case seen from inside: the previous run was killed mid-room, its lease is real until Redis lets it go, and the new run cannot renew a holder value it does not have, so the honest thing is to wait exactly as it would for a stranger's. Doing that on the server, with the peer held connecting, is also what bounds failover to one ttl rather than a ttl plus a client backoff.

▼ 3. The fence compares a random claim id in `insert ... select ... where fence_claim is not distinct from $9 for update, and claim uses strictly <`. Why not read the claim first in TypeScript and then insert; why a claim id and not "token at least the largest seen"; why `<` and not `<=`; and what does `for update` buy?

Read-then-write has an `await` between the two, and a process can be frozen for a minute inside an `await` : A reads its claim (valid), freezes, B claims and reads the journal, A wakes and inserts a row one past what B read. The naive control in `fence.int` performs exactly that and asserts the zombie row is in the journal. In one statement the check and the write cannot be separated by a pause. A token high-water mark fails for two reasons specific to this repo: the counter lives in Redis and a flush restarts it, so the floor from `fence_token` is needed to keep tokens rising at all; and even then one token can be issued twice (a node paused between acquire and claim, then a flush that reissues the number), so the thing an append carries is an id of *this one opening*, and the loser of two openings under an equal token cannot write. `<` rather than `<=` is the same case at the claim: with `<=` both openings under token 7 would claim and both would hold a valid claim id. `for update` takes a row lock on the documents row so that an append and a claim serialise: the claim waits for an in-flight insert to commit (and the new owner's replay then contains the row), or the claim commits first and the blocked append re-evaluates its `where` against the updated row and inserts nothing. Without the lock the two could interleave at the statement level and the "waits, then is refused" test would not hold.

Recall first

Look away. Name the three Lua scripts and what each one checks before it acts. Then name the four things a node does when its lease is lost or its append is fenced. Then check.

▼ Check yourself

`ACQUIRE` checks "nobody holds it" and then lifts the counter to the floor, `INCR` s a token and `SET` s with `PX` , all in one script; `RENEW` checks "the value is still exactly mine" and `PEXPIRE` s; `RELEASE` checks the same and `DEL` s. On a lost lease or a `Fenced` append, `drop` : marks the room `lost` , stops its snapshots, forgets the room (so the next peer asks Redis again), and closes every socket with 4409, which peer-client treats as "ask `/session` again".

Then test it

1. Ten peers of a new document join at once, five to each node. What token does the winner hold, and why does that number mean anything?

1: a token is issued only on the branch of the one script that also does the `SET`, so losers cost nothing and "the counter moved by one" is a checkable fact

0 until the first renewal

10: every acquirer increments the counter

Whichever `INCR` ran first; it varies

The join-race test asserts `token: 1` after the race and `2` after the room closes and is taken again.

2. A renewal is sent at 300 ms and answered at 1100 ms (ttl 1000). Until when does the holder trust the lease?

2100: reply + ttl

1200: sent + 0.9 ttl, because Redis started its countdown when the command arrived, after 300

2000: reply + 0.9 ttl

1300: sent + ttl

Counting from the reply would trust the lease for a whole round trip longer than Redis keeps it. The test with the slow renewal is the mutation check.

3. The owner of a room stops beating. What does the other node do when a peer of that room arrives?

Holds the peer connecting and polls every ttl/10 until the lease expires, then takes the room; it never deletes or overwrites the dead lease

Takes the lease at once: no heartbeat means dead

Opens a second room and merges later

Refuses with 4409 and lets the peer retry on its own backoff

"Stopped beating" is not "stopped writing" (a paused node wakes). Liveness routes; the lease alone owns.

4. Why does a room that is fenced *drop* (4409 to every peer) rather than go read-only and heal, as it does when Postgres is down?

Because read-only mode does not work with two nodes

A fenced room will never be allowed to write again: a newer owner holds the claim, so healing would ping for ever; its peers belong with the new owner

It does go read-only; the drop is only for a lost lease

Because the snapshot would otherwise be lost

`watchFence` tells the server once; `onReadOnly` checks `fenced` so the heal loop never starts.

5. Why is the fence check inside the `insert ... select ... where fence_claim is not distinct from $9 for update` statement rather than a `select` in TypeScript before the insert?

TypeScript cannot compare uuids

It saves one round trip

Postgres refuses two statements on the same row

A process can be frozen inside the `await` between the two: it reads a valid claim, another node claims and reads the journal, and it wakes and inserts a row the new owner never sees; one statement cannot be split by a pause

The naive control in `fence.int` performs the interleaving and asserts the zombie's row IS in the journal. Drill 2 plants it.

6. Redis is flushed while both nodes run. Why do tokens keep rising, and why is that not enough on its own?

The counter is persisted with AOF

They do not; the nodes restart

Postgres issues the tokens

Every acquire passes a floor read from `documents.fence_token`, and the script lifts the counter to it before `INCR`; but one token can still be issued twice (paused between acquire and claim, then a flush), so the append compares an exact claim id and the claim uses strictly <

The lease integration test deletes the counter mid-test and asserts 43 follows 42; `fence.int` asserts a second claim under the same token is refused.

7. What does `for update` add to the append statement?

It makes the insert faster

Nothing: it is required syntax for `insert ... select`

It locks the `op_journal` table

A row lock on the `documents` row, so an append and a claim serialise: the claim waits for an in-flight insert, or the blocked append re-checks its `where` against the claimed row and inserts nothing

The test "an append racing an uncommitted claim waits for it, then is refused" holds a second connection's transaction open for 300 ms and asserts the append has not settled.

8. What does this design give up, in CAP terms, and why is that the right side?

Consistency: peers may briefly see different documents

Availability for up to one ttl after an owner dies: nobody takes the room until the lease expires, and peers hold their edits; two rooms for one document would be a divergence lesson 2's design cannot repair, while peer-client already absorbs a pause

Nothing: Redis makes it both

Durability: ops made during the ttl are dropped

`LEASE_TTL_MS` is the dial; the chaos run's budget is expressed in ttl so the test moves with it.

Next: Lesson 8, after epic 8. Auth, tenancy, RBAC and audit: a real identity replaces the dev header, roles decide what a member may do in every route and in the room, and every change is written down with who made it.